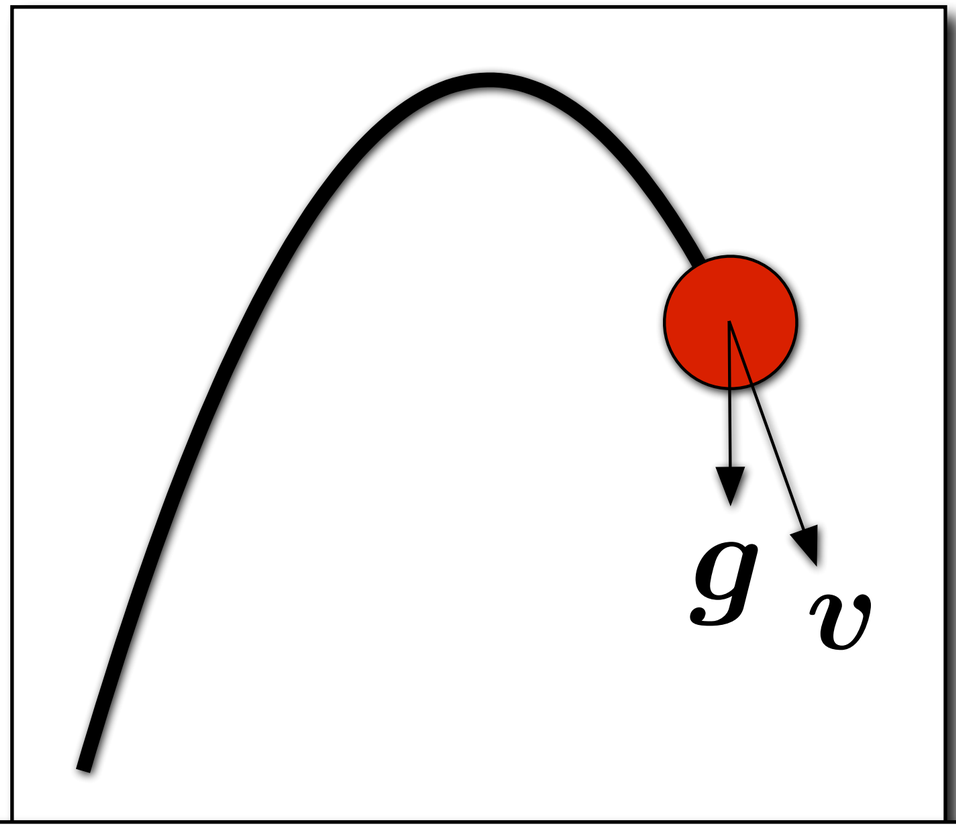
Section 17:

Introduction to Physical Simulation

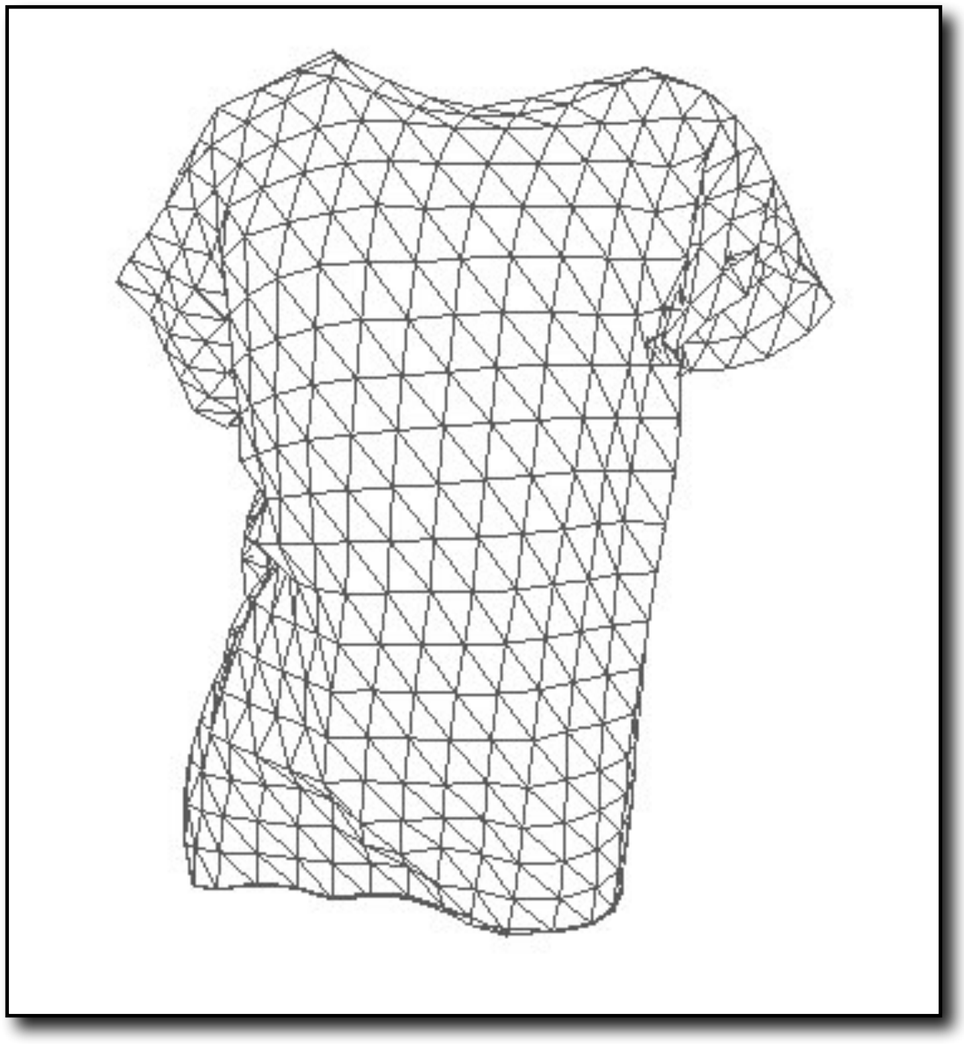
Computer Graphics and Imaging
UC Berkeley CS184/284A

Physically Based Animation

Generate motion of objects using numerical simulation



$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \Delta t \boldsymbol{v}^t + \frac{1}{2}(\Delta t)^2 \boldsymbol{a}^t$$



Example: Cloth Simulation



Example: Cloth Simulation



Example: Fluids



Macklin and Müller, Position Based Fluids TOG 2013

Example: Fluids



Macklin and Müller, Position Based Fluids TOG 2013

Particle Systems

Single particles are very simple

Large groups can produce interesting effects

Supplement basic ballistic rules

- Gravity
- Friction, drag
- Collisions
- Force fields
- Springs
- Interactions
- Others...



Karl Sims, SIGGRAPH 1990


PARTICLE DREAMS
Karl Sims
Optomystic


PARTICLE DREAMS
Karl Sims
Optomystic

Particle Systems

- Single particles are very simple
- Large groups can produce interesting effects
- Supplement basic ballistic rules
  - Collisions
  - Interactions
  - Force fields
  - Springs
  - Others...

Feldman, Klingner, O'Brien, SIGGRAPH 2005

Particle Systems

- Single particles are very simple
- Large groups can produce interesting effects
- Supplement basic ballistic rules
  - Collisions
  - Interactions
  - Force fields
  - Springs
  - Others...

Feldman, Klingner, O'Brien, SIGGRAPH 2005

Basic Particles

- Basic governing equation
  - $\boldsymbol{f}$ is a sum of a number of things

$$\ddot{\boldsymbol{x}} = \frac{1}{m}\boldsymbol{f}$$

    - Gravity: constant downward force proportional to mass
    - Simple drag: force proportional to negative velocity
    - Particle interactions: particles mutually attract and/or repell
      - Beware $O(n^2)$ complexity!
  - Force fields
  - Wind forces
  - User interaction

Dot Notation for Derivatives

If x is a vector for the position of a point of interest, we will use dot notation for velocity and acceleration:

$$x$$

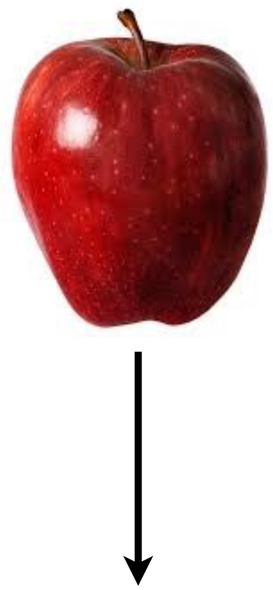
$$\dot{x} = v$$

$$\ddot{x} = a$$

Simple Gravity

Gravity at earth's surface due to earth

- F = –mg
- m is mass of object
- g is gravitational acceleration, g = –9.8m/s²



$$F_g = -mg$$

$$g = (0, 0, -9.8)\,\mathrm{m/s^2}$$

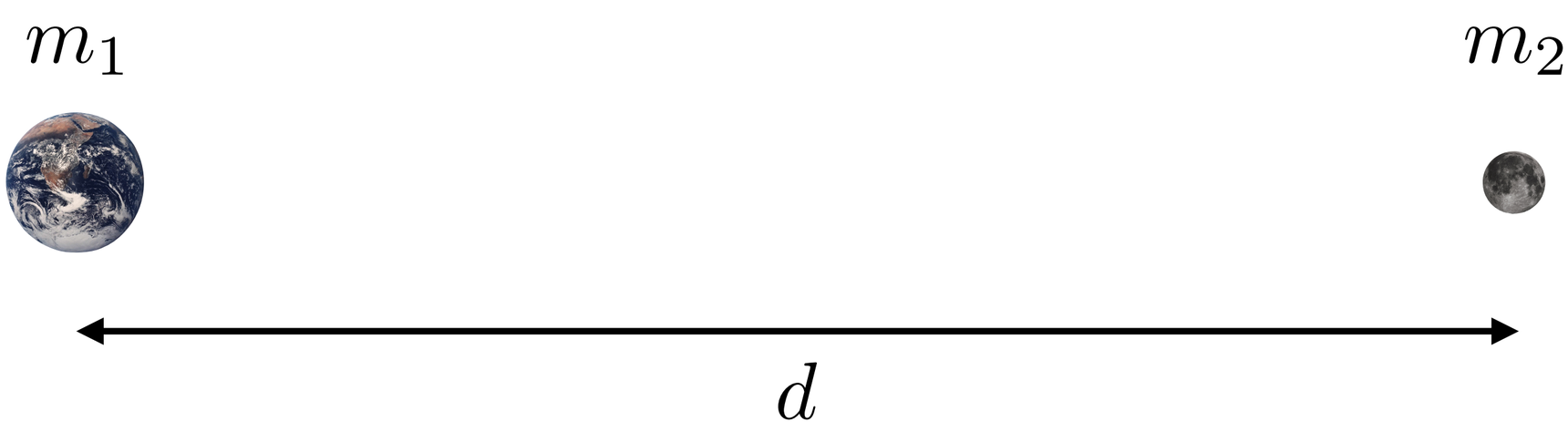
Gravitational Attraction

Newton's universal law of gravitation

- Gravitational pull between particles

$$F_g = G\frac{m_1 m_2}{d^2}$$

$$G = 6.67428 \times 10^{-11}\,\mathrm{Nm^2kg^{-2}}$$



Example: Galaxy Simulation



Disk galaxy simulation, NASA Goddard

Example: Galaxy Simulation



Disk galaxy simulation, NASA Goddard

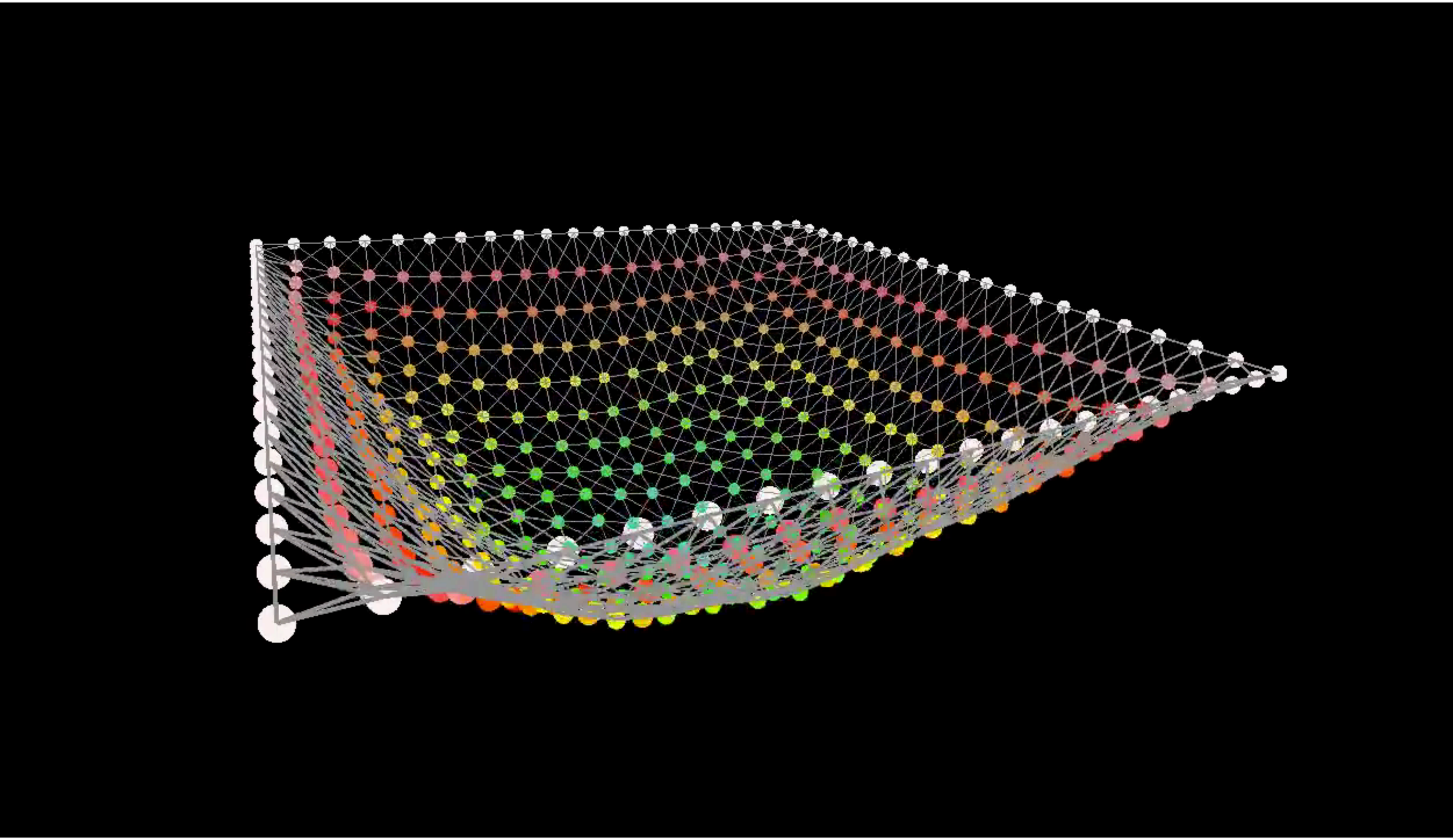
Rendering Particles

Rendering styles

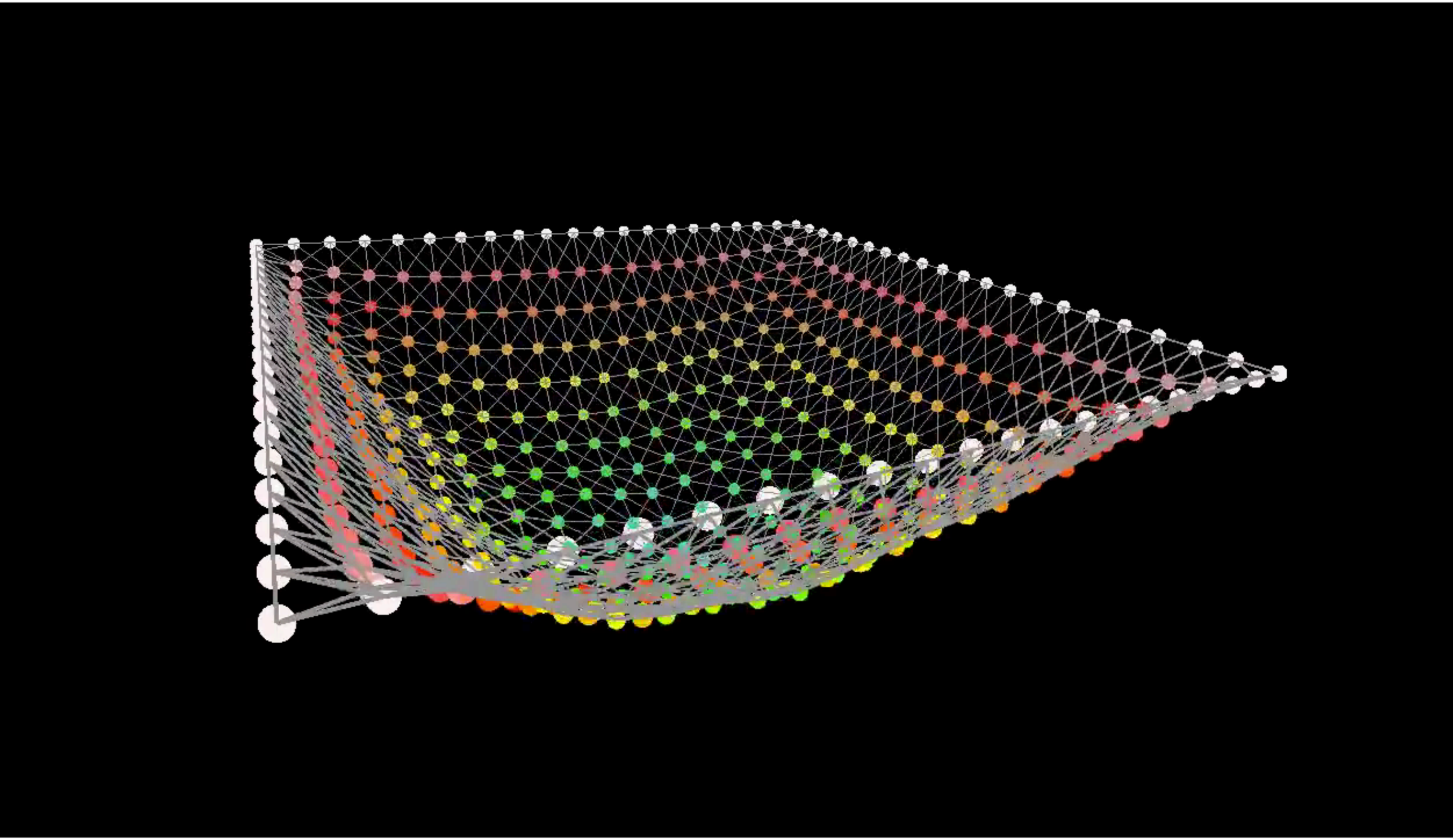
- Points
- Polygons
- Shapes
- Trails
- Volume reconstruction

Mass + Spring Systems:
Example of Modeling a Dynamical System

Example: Mass Spring Mesh



Example: Mass Spring Mesh



Example: Mass Spring Rope



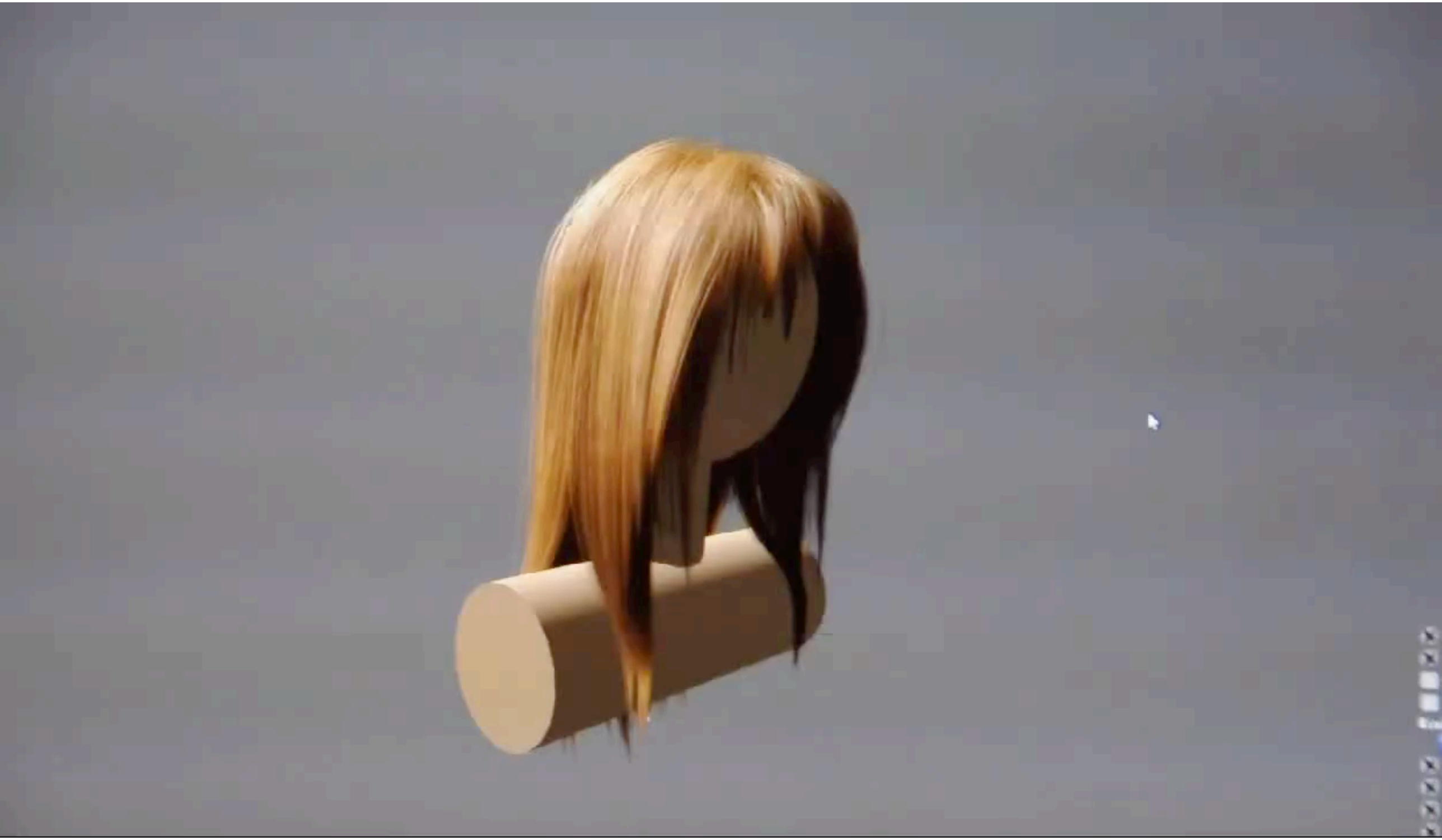
Credit: Elizabeth Labelle, https://youtu.be/Co8enp8CH34

Example: Mass Spring Rope

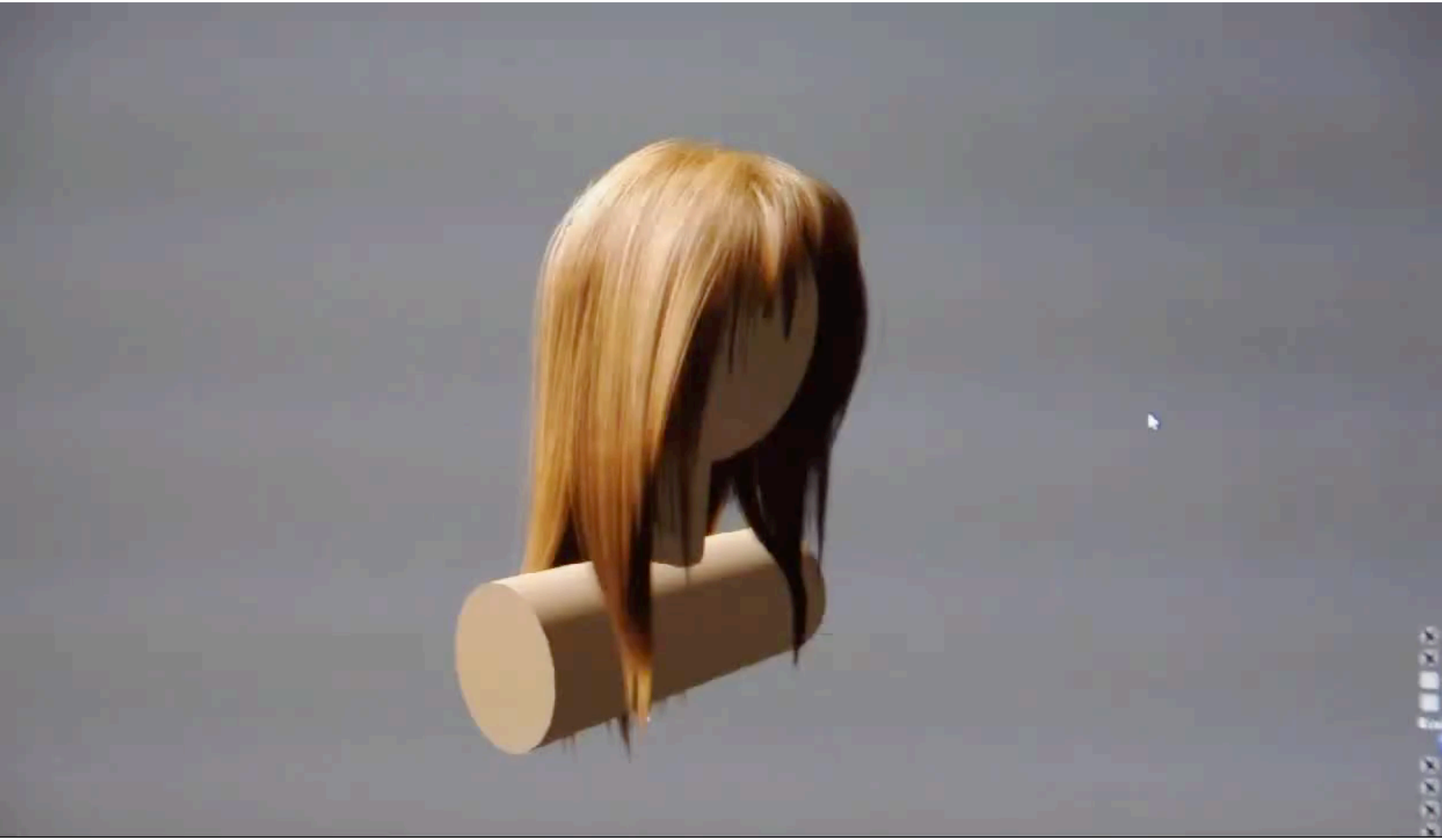


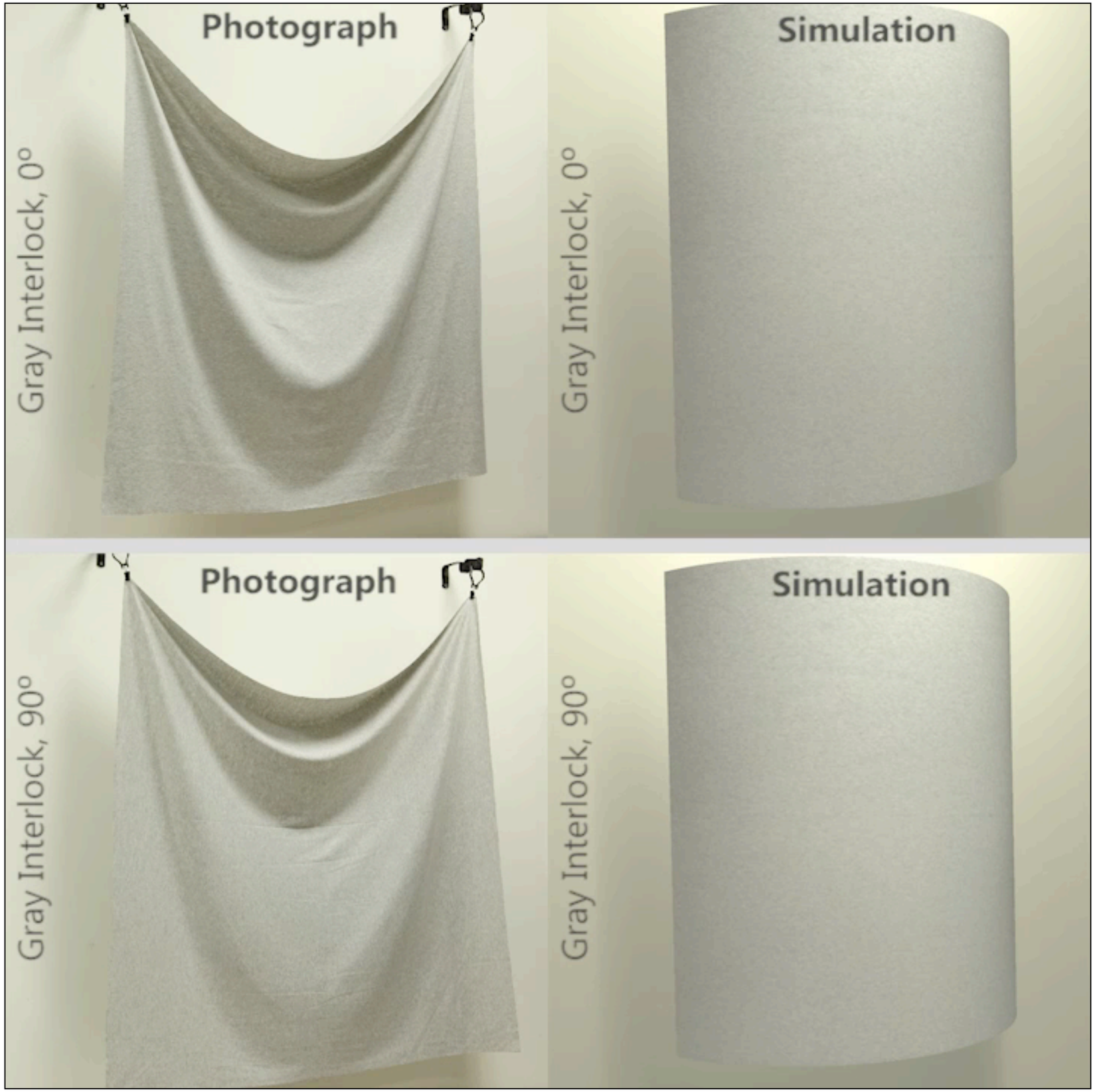
Credit: Elizabeth Labelle, https://youtu.be/Co8enp8CH34

Example: Hair

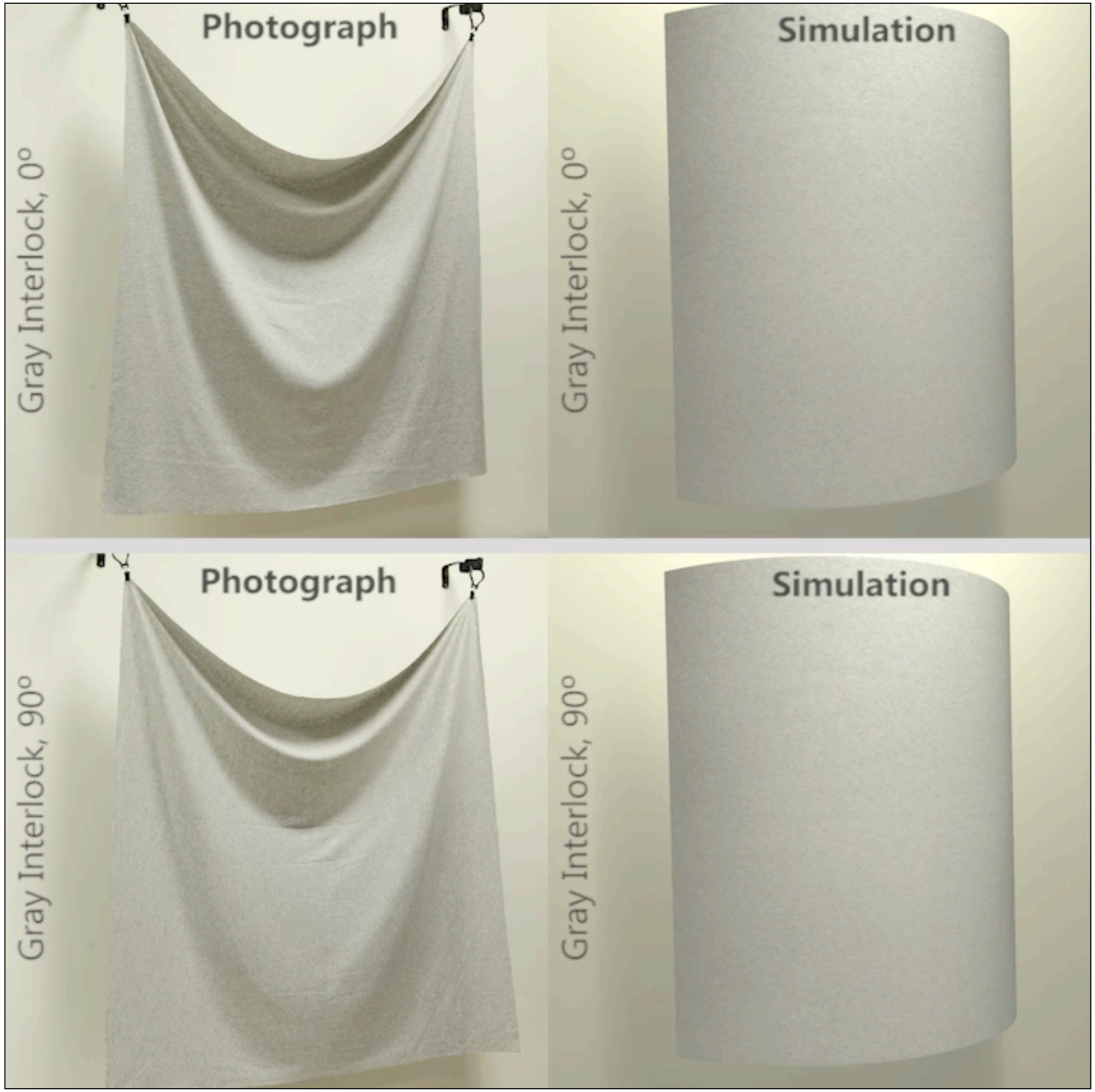


Example: Hair





Huamin Wang, Ravi Ramamoorthi, and James F. O'Brien. "Data-Driven Elastic Models for Cloth: Modeling and Measurement". *ACM Transactions on Graphics*, 30(4):71:1–11, July 2011. Proceedings of ACM SIGGRAPH 2011, Vancouver, BC Canada.



Huamin Wang, Ravi Ramamoorthi, and James F. O'Brien. "Data-Driven Elastic Models for Cloth: Modeling and Measurement". *ACM Transactions on Graphics*, 30(4):71:1–11, July 2011. Proceedings of ACM SIGGRAPH 2011, Vancouver, BC Canada.

A Simple Spring

Idealized spring



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

$$\boldsymbol{f}_{b \to a} = -\boldsymbol{f}_{a \to b}$$

Force pulls points together

Strength proportional to displacement (Hooke's Law)

k_s is a spring coefficient: stiffness

A Simple Spring

Idealized spring



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

$$\boldsymbol{f}_{b \to a} = -\boldsymbol{f}_{a \to b}$$

Force pulls points together

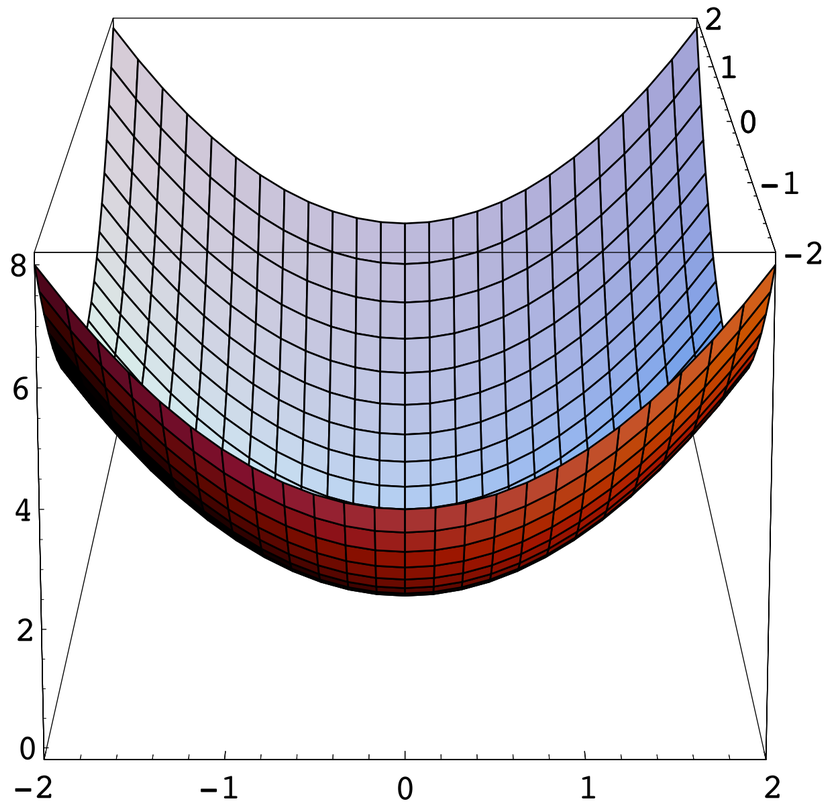
Strength proportional to displacement (Hooke's Law)

k_s is a spring coefficient: stiffness

Problem: this spring wants to have zero length

A Simple Spring

Energy potential

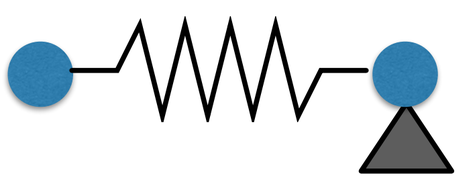


$$E = 1/2\ k_S(\boldsymbol{b} - \boldsymbol{a}) \cdot (\boldsymbol{b} - \boldsymbol{a})$$

$$\boldsymbol{f}_{a \rightarrow b} = k_S(\boldsymbol{b} - \boldsymbol{a})$$

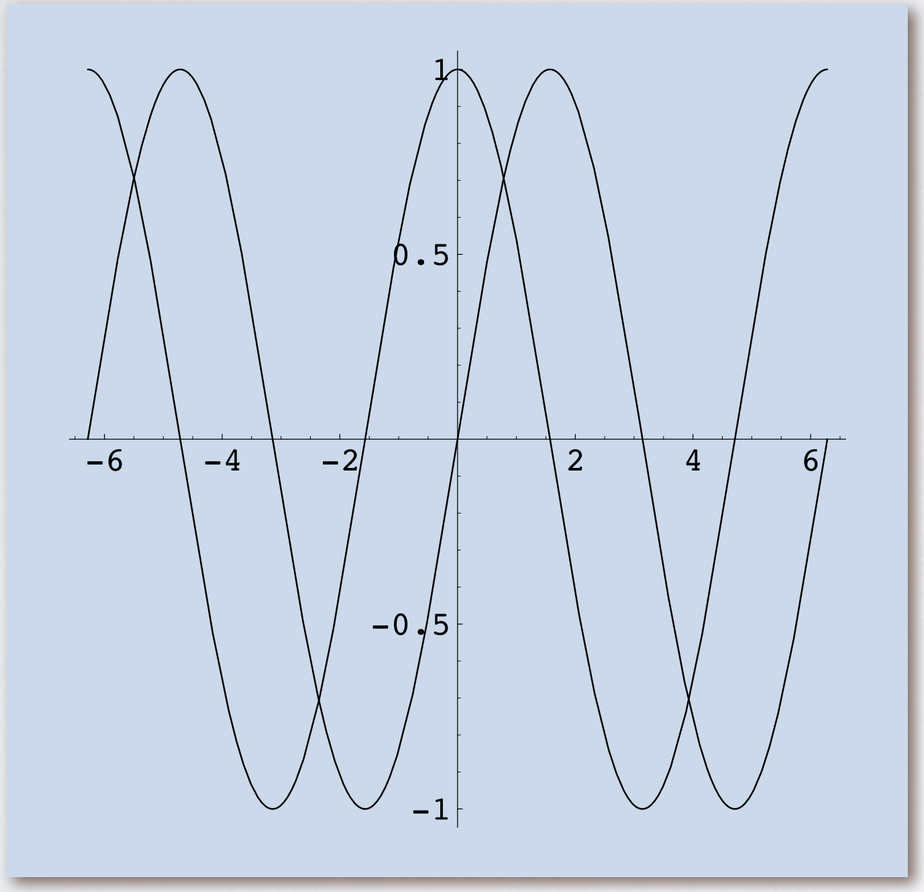
$$\boldsymbol{f}_{b \rightarrow a} = -\boldsymbol{f}_{a \rightarrow b}$$

$$\boldsymbol{f}_a = -\nabla a E = -\left[\frac{\partial E}{\partial a_x}, \frac{\partial E}{\partial a_y}, \frac{\partial E}{\partial a_z}\right]$$



A Simple Spring

- Energy potential: kinetic ***vs*** elastic



$$E = 1/2\ k_s(\boldsymbol{b} - \boldsymbol{a}) \cdot (\boldsymbol{b} - \boldsymbol{a})$$

$$E = 1/2\ m(\dot{\boldsymbol{b}} - \dot{\boldsymbol{a}}) \cdot (\dot{\boldsymbol{b}} - \dot{\boldsymbol{a}})$$



Non-Zero Length Spring

Spring with non-zero rest length



$$\boldsymbol{f}_{a \rightarrow b} = k_S \frac{\boldsymbol{b} - \boldsymbol{a}}{||\boldsymbol{b} - \boldsymbol{a}||} (||\boldsymbol{b} - \boldsymbol{a}|| - l)$$

Rest length

Non-Zero Length Spring

Spring with non-zero rest length



$$\boldsymbol{f}_{a \rightarrow b} = k_S \frac{\boldsymbol{b} - \boldsymbol{a}}{||\boldsymbol{b} - \boldsymbol{a}||} (||\boldsymbol{b} - \boldsymbol{a}|| - l)$$

Rest length

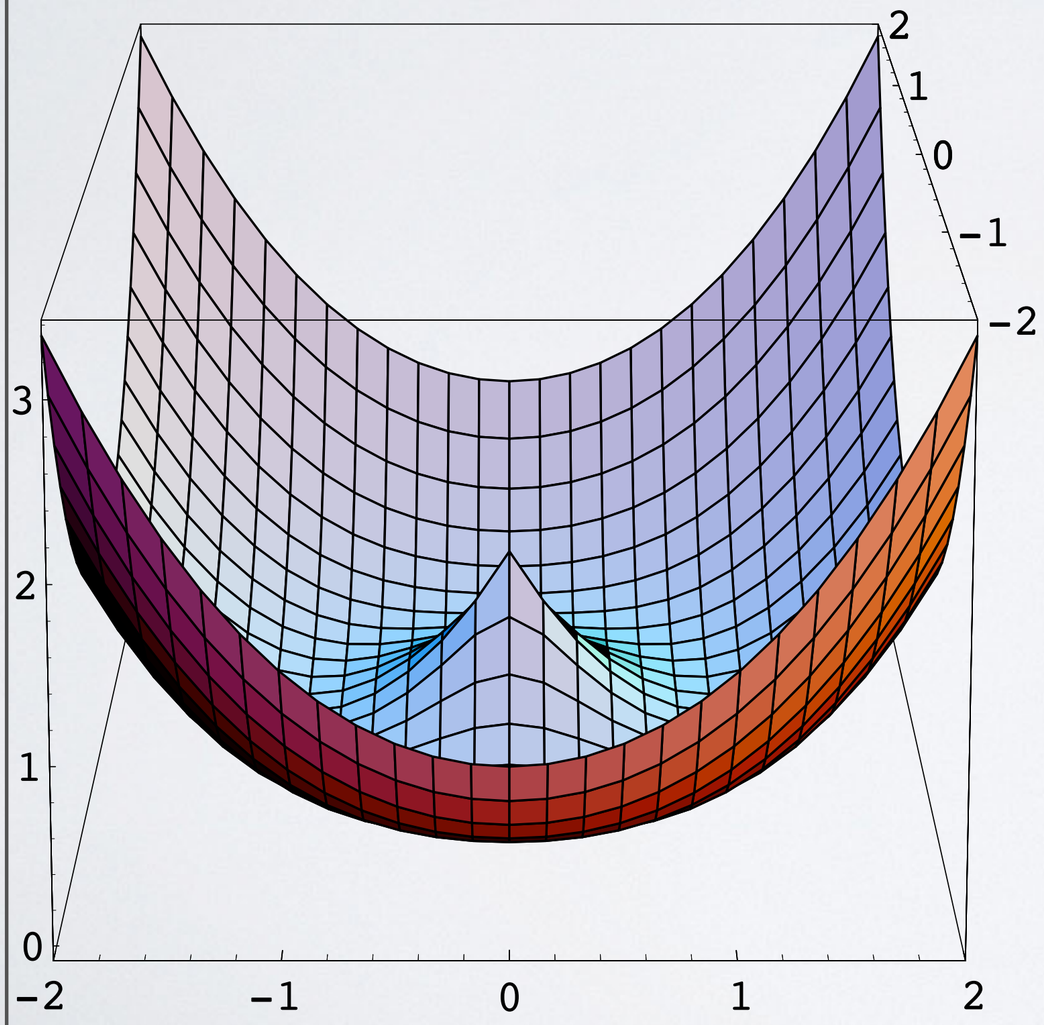
Problem: oscillates forever

Non-Zero Length Springs



$$\boldsymbol{f}_{a \to b} = k_S \frac{\boldsymbol{b} - \boldsymbol{a}}{||\boldsymbol{b} - \boldsymbol{a}||} \left(||\boldsymbol{b} - \boldsymbol{a}|| - l \right)$$

Rest length



$$E = k_S \left(||\boldsymbol{b} - \boldsymbol{a}|| - l \right)^2$$

Comments on Springs

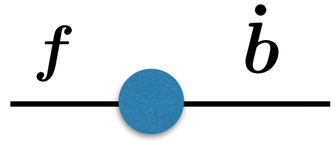
Springs with zero rest length are linear

Springs with non-zero rest length are nonlinear

- Force magnitude linear w/ displacement (from rest length)
- Force direction is non-linear
- Singularity at $||\boldsymbol{b} - \boldsymbol{a}|| = 0$

Simple Motion Damping

Simple motion damping

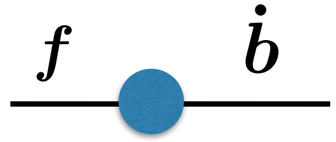


$$\boldsymbol{f} = -k_d \dot{\boldsymbol{b}}$$

- Behaves like viscous drag on motion
- Slows down motion in the direction of motion
- k_d is a damping coefficient

Simple Motion Damping

Simple motion damping



$$\boldsymbol{f} = -k_d \dot{\boldsymbol{b}}$$

- Behaves like viscous drag on motion
- Slows down motion in the direction of motion
- k_d is a damping coefficient

Problem: slows down *all* motion

- Want a rusty spring's oscillations to slow down, but should it also fall to the ground more slowly?

Internal Damping for Spring

Damp only the internal, spring-driven motion

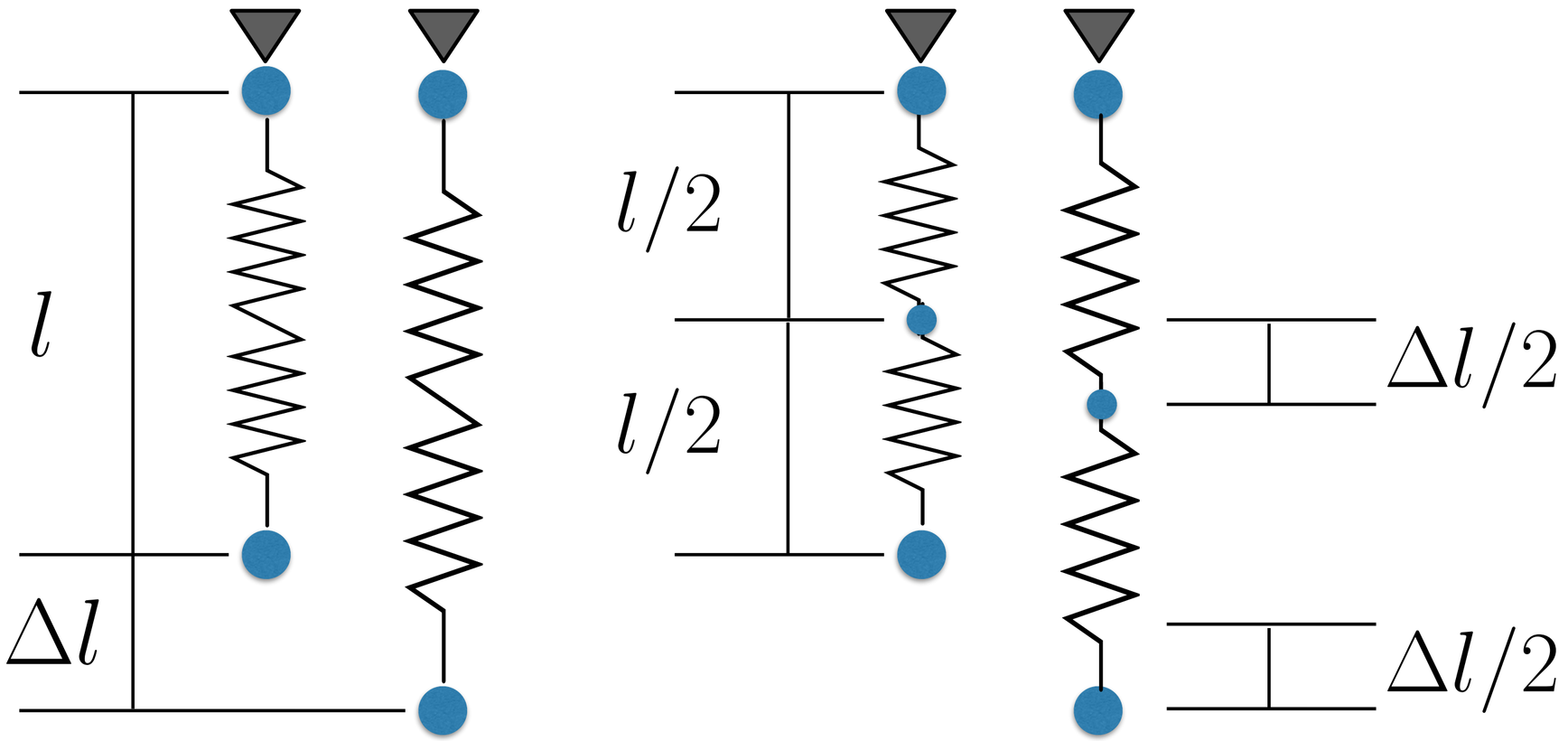


$$\boldsymbol{f}_a = -k_d \frac{\boldsymbol{b}-\boldsymbol{a}}{||\boldsymbol{b}-\boldsymbol{a}||}(\dot{\boldsymbol{b}}-\dot{\boldsymbol{a}}) \cdot \frac{\boldsymbol{b}-\boldsymbol{a}}{||\boldsymbol{b}-\boldsymbol{a}||}$$

- Viscous drag only on change in spring length
  - Won’t slow group motion for the spring system (e.g. global translation or rotation of the group)

Spring Constants

Consider two “resolutions” to model a single spring



Problem: constant k_s produces different force on bottom spring for these two different discretizations

Spring Constants

Spring Constants

Problem: constant k_s gives inconsistent results with different discretizations of our spring/mass structures

- E.g. 10x10 vs 20x20 mesh for cloth simulation would give different results, and we want them to be the same, just higher level of detail

Spring Constants

Problem: constant k_s gives inconsistent results with different discretizations of our spring/mass structures

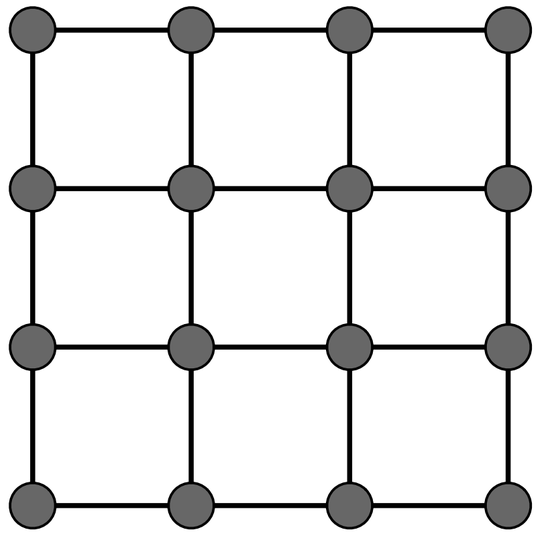
- E.g. 10x10 vs 20x20 mesh for cloth simulation would give different results, and we want them to be the same, just higher level of detail

Solution:

- Change in length is not what we want to measure
- We want to consider the strain = change in length as fraction of original length $$\epsilon = \frac{\Delta l}{l_0}$$
- Implementation 1: divide spring force by spring length
- Implementation 2: normalize k_s by spring length

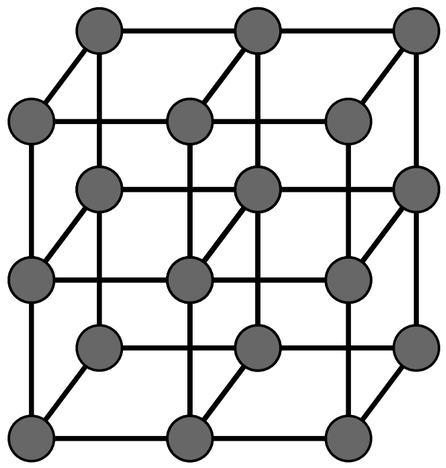
Structures from Springs

Sheets



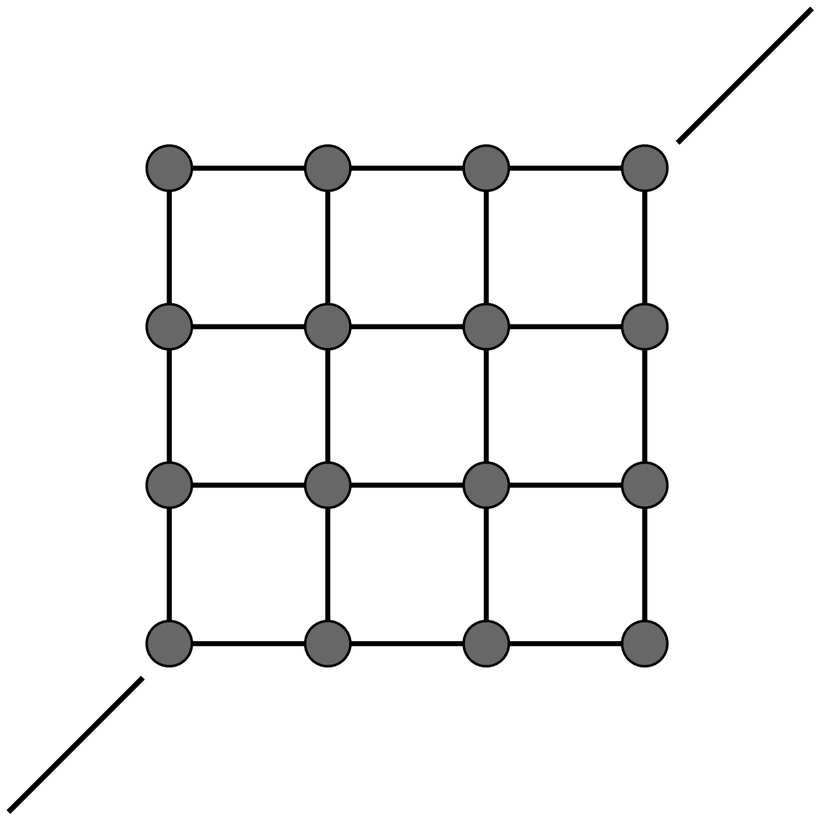
Blocks

Others



Structures from Springs

Behavior is determined by structure linkages

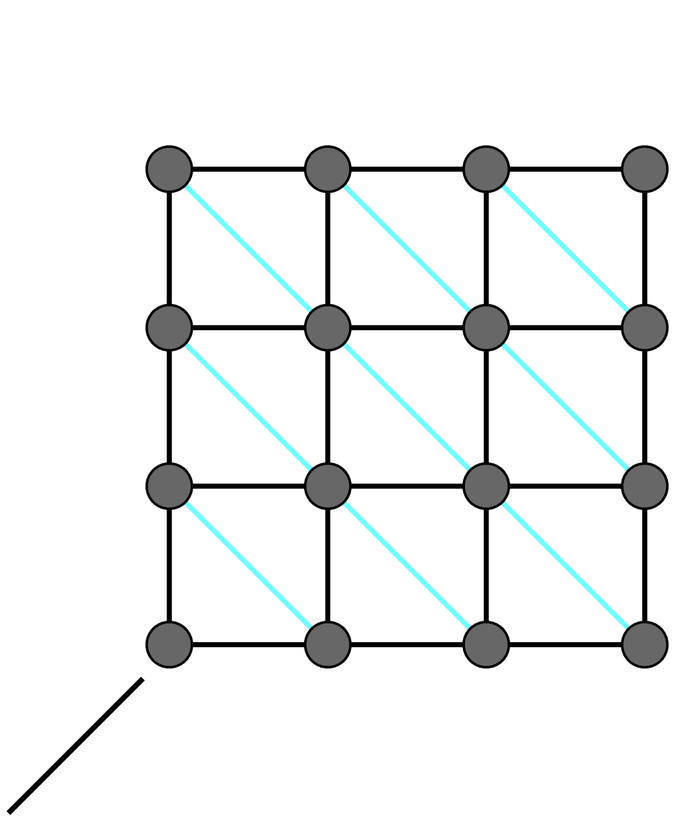


This structure will not resist shearing

This structure will not resist out-of-plane bending...

Structures from Springs

Behavior is determined by structure linkages

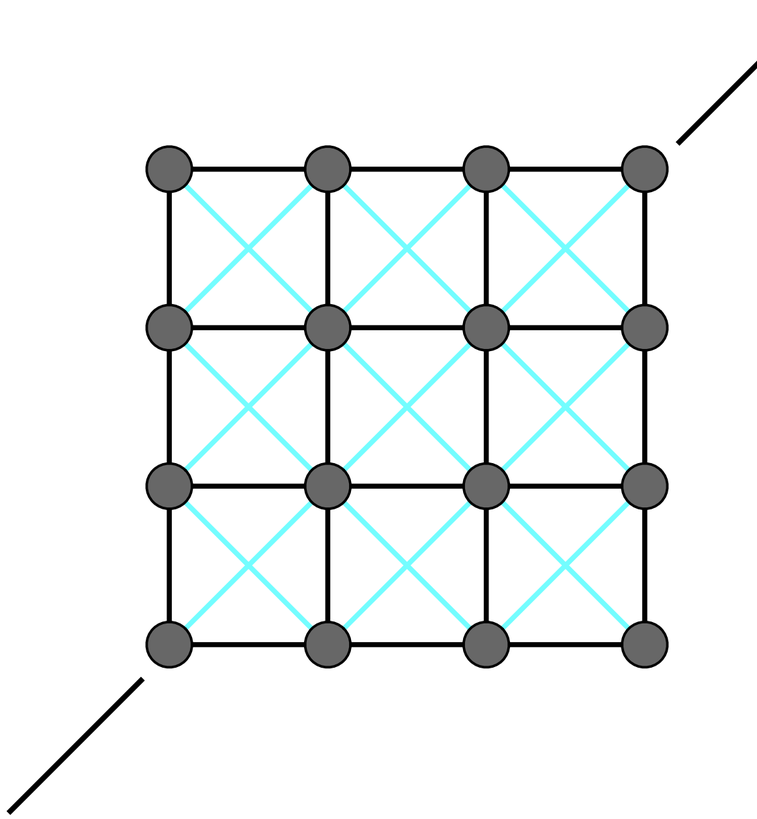


This structure will resist shearing but has anisotropic bias

This structure will not resist out-of-plane bending either...

Structures from Springs

Behavior is determined by structure linkages

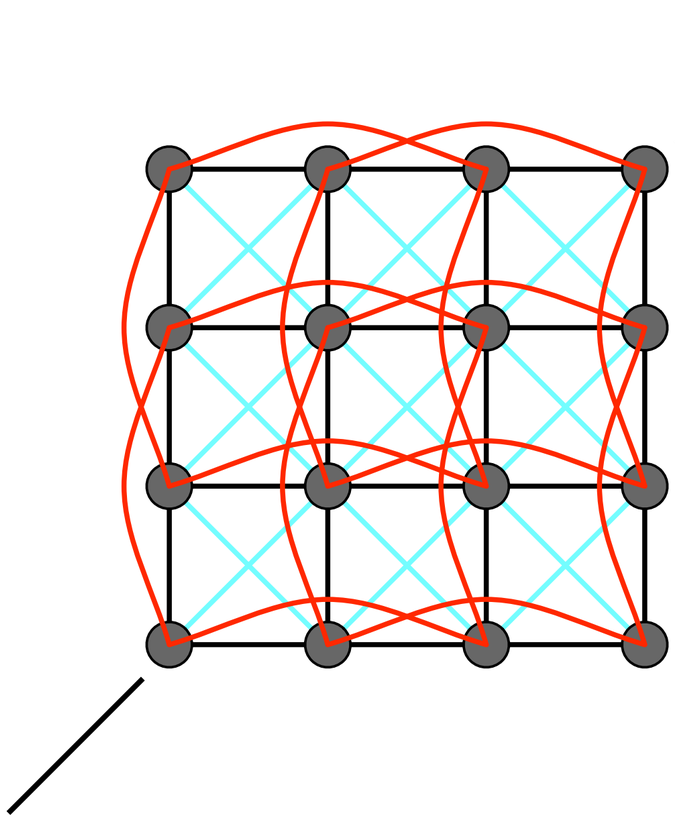


This structure will resist shearing.
Less directional bias.

This structure will not resist out-of-plane bending either...

Structures from Springs

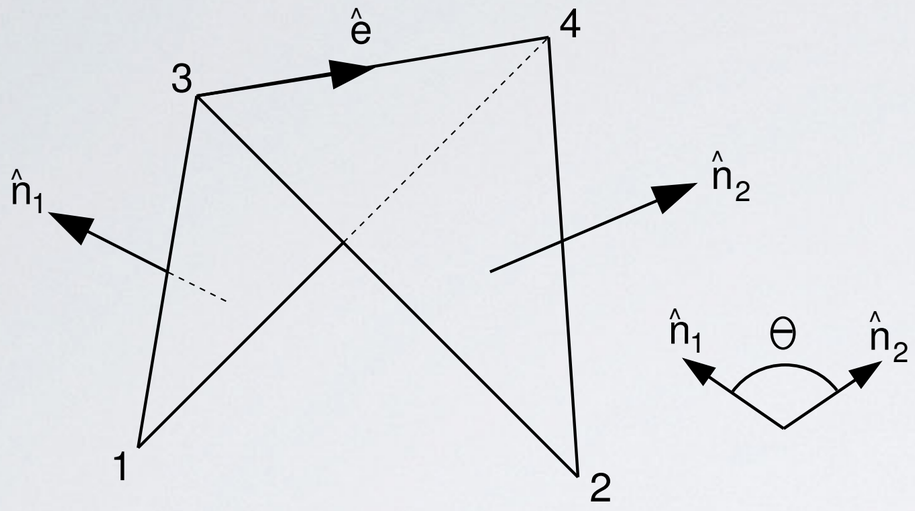
They behave like what they are (obviously!)



This structure will resist shearing.
Less directional bias.

This structure will resist out-of-plane bending
Red springs should be much weaker

Edge Springs



From Bridson ***et al.,*** 2003

$$u_1 = |E|\frac{N_1}{|N_1|^2} \qquad u_2 = |E|\frac{N_2}{|N_2|^2}$$

$$u_3 = \frac{(x_1 - x_4)\cdot E}{|E|}\frac{N_1}{|N_1|^2} + \frac{(x_2 - x_4)\cdot E}{|E|}\frac{N_2}{|N_2|^2}$$

$$u_4 = -\frac{(x_1 - x_3)\cdot E}{|E|}\frac{N_1}{|N_1|^2} - \frac{(x_2 - x_3)\cdot E}{|E|}\frac{N_2}{|N_2|^2}$$

$$F_i^e = k^e \frac{|E|^2}{|N_1| + |N_2|}\sin(\theta/2)\, u_i$$

Strain Limiting



Huamin Wang, James F. O'Brien, and Ravi Ramamoorthi. "**Multi-Resolution Isotropic Strain Limiting**". In *Proceedings of ACM SIGGRAPH Asia 2010*, pages 160:1–10, December 2010.

Strain Limiting



Huamin Wang, James F. O'Brien, and Ravi Ramamoorthi. "**Multi-Resolution Isotropic Strain Limiting**". In *Proceedings of ACM SIGGRAPH Asia 2010*, pages 160:1–10, December 2010.

Adaptive Simulation





Adaptive Simulation





Example: Mass Spring Dress + Character



Example: Mass Spring Dress + Character



Aside: FEM (Finite Element Method) Instead of Springs





Aside: FEM (Finite Element Method) Instead of Springs





Combating Instability

Euler's Method

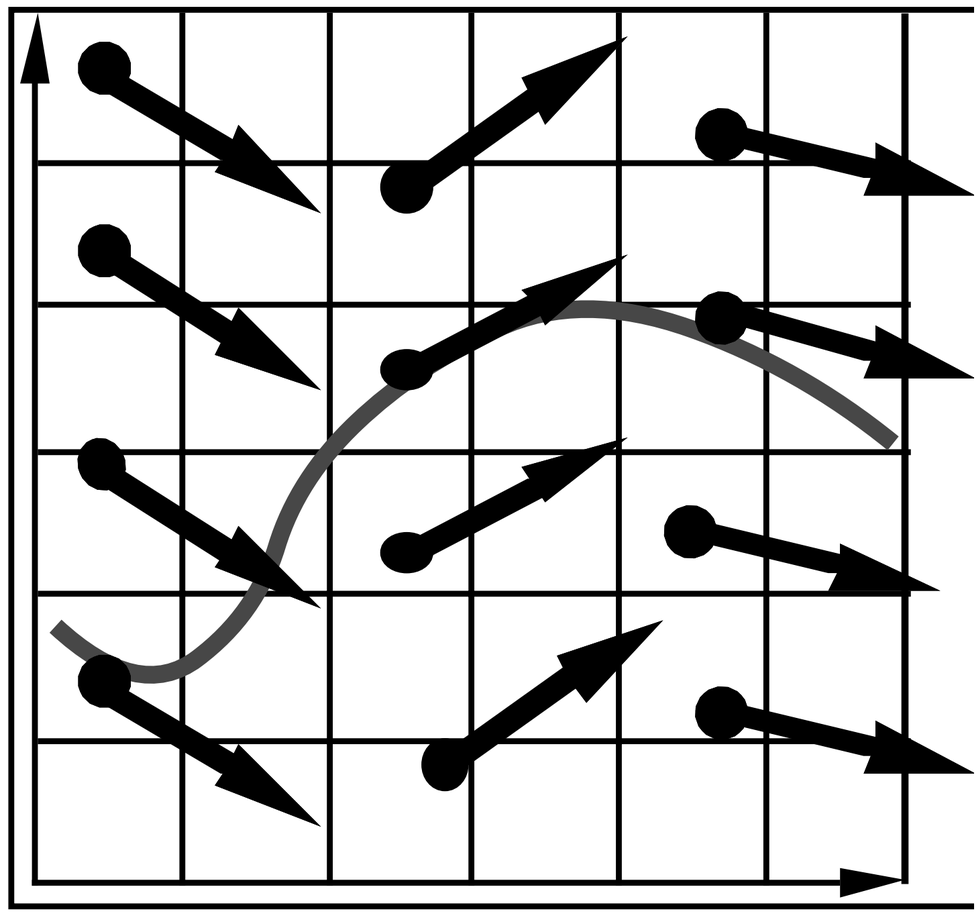
Euler's Method (a.k.a. Forward Euler, Explicit)

- Simple iterative method
- Commonly used
- Very inaccurate
- Most often goes unstable

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t\, \dot{\boldsymbol{x}}^{t}$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \dot{\boldsymbol{x}}^{t} + \Delta t\, \ddot{\boldsymbol{x}}^{t}$$

Single Particle Simulation

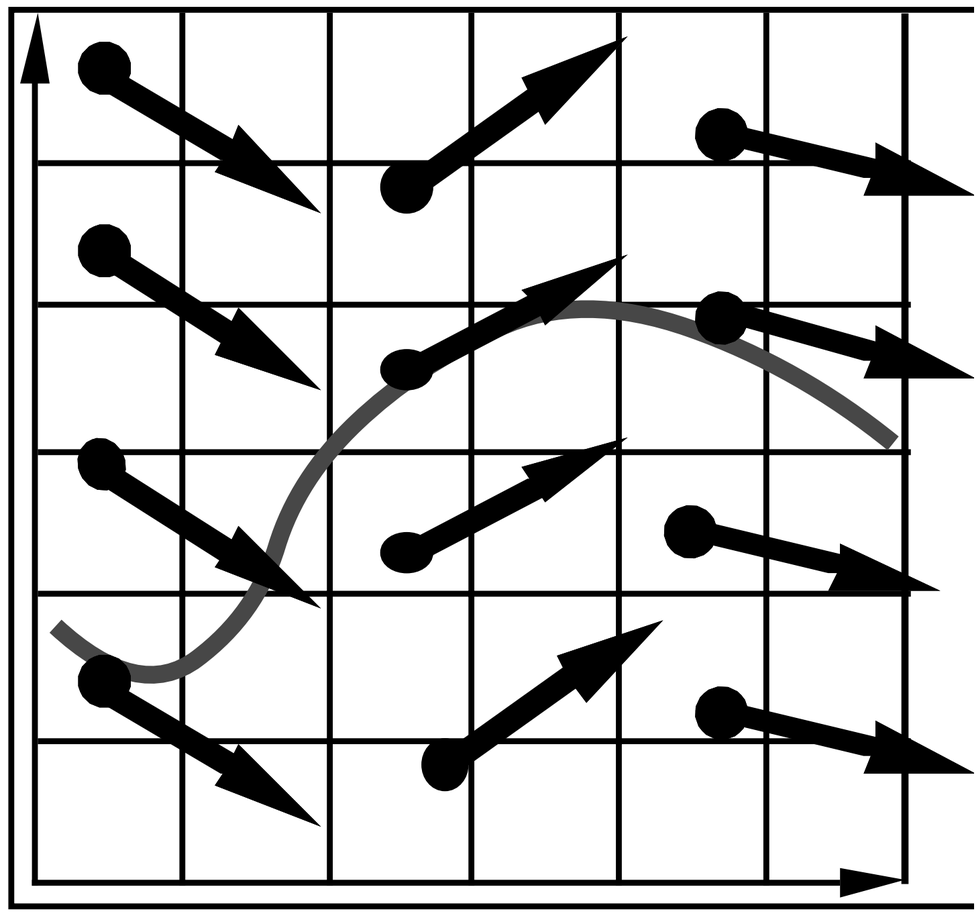


Witkin and Baraff

Single Particle Simulation

First study motion of a single particle

- Later, generalize to a multitude of particles



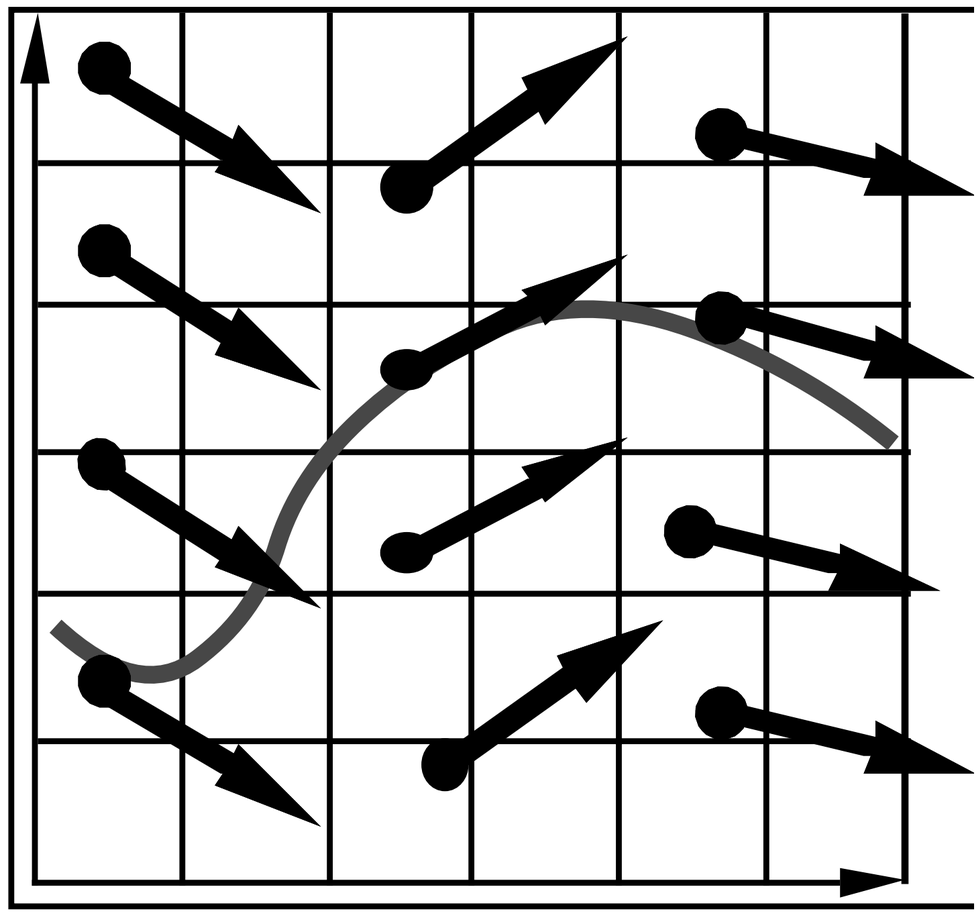
Witkin and Baraff

Single Particle Simulation

First study motion of a single particle

- Later, generalize to a multitude of particles

To start, assume motion of particle determined by a velocity vector field that is a function of position and time:



Witkin and Baraff

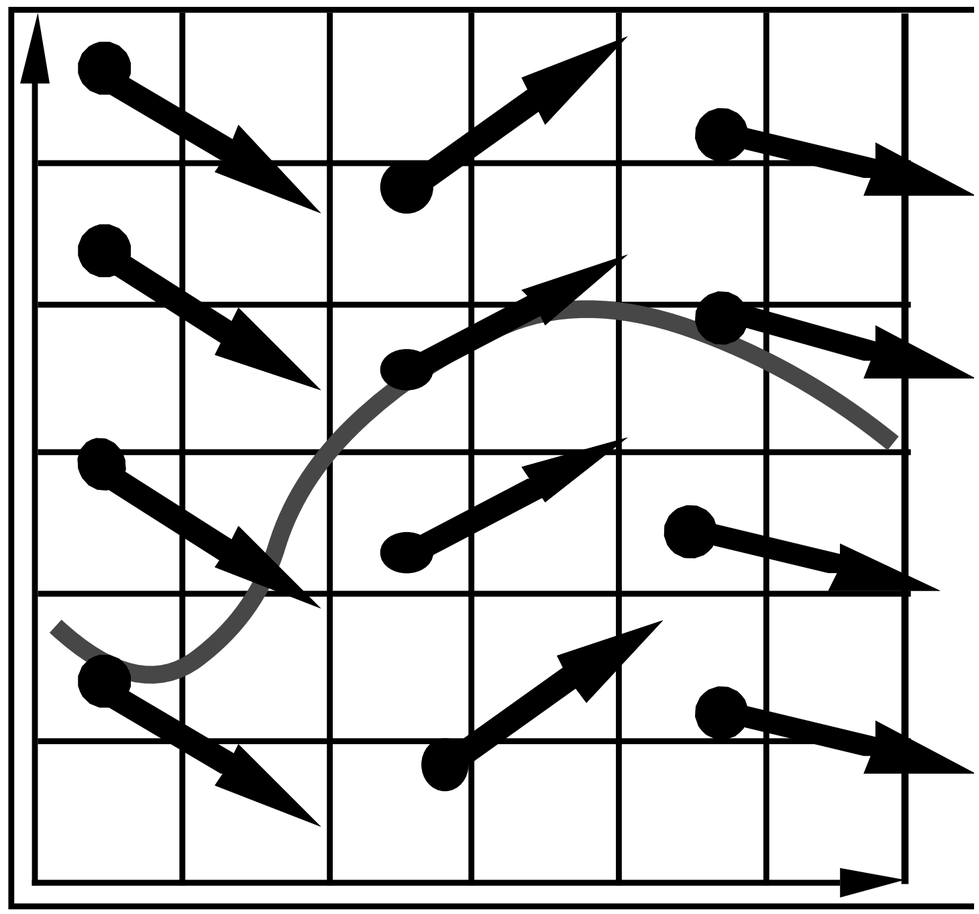
Single Particle Simulation

First study motion of a single particle

- Later, generalize to a multitude of particles

To start, assume motion of particle determined by a velocity vector field that is a function of position and time: $v(x,t)$

In example v and x are 2D



Witkin and Baraff

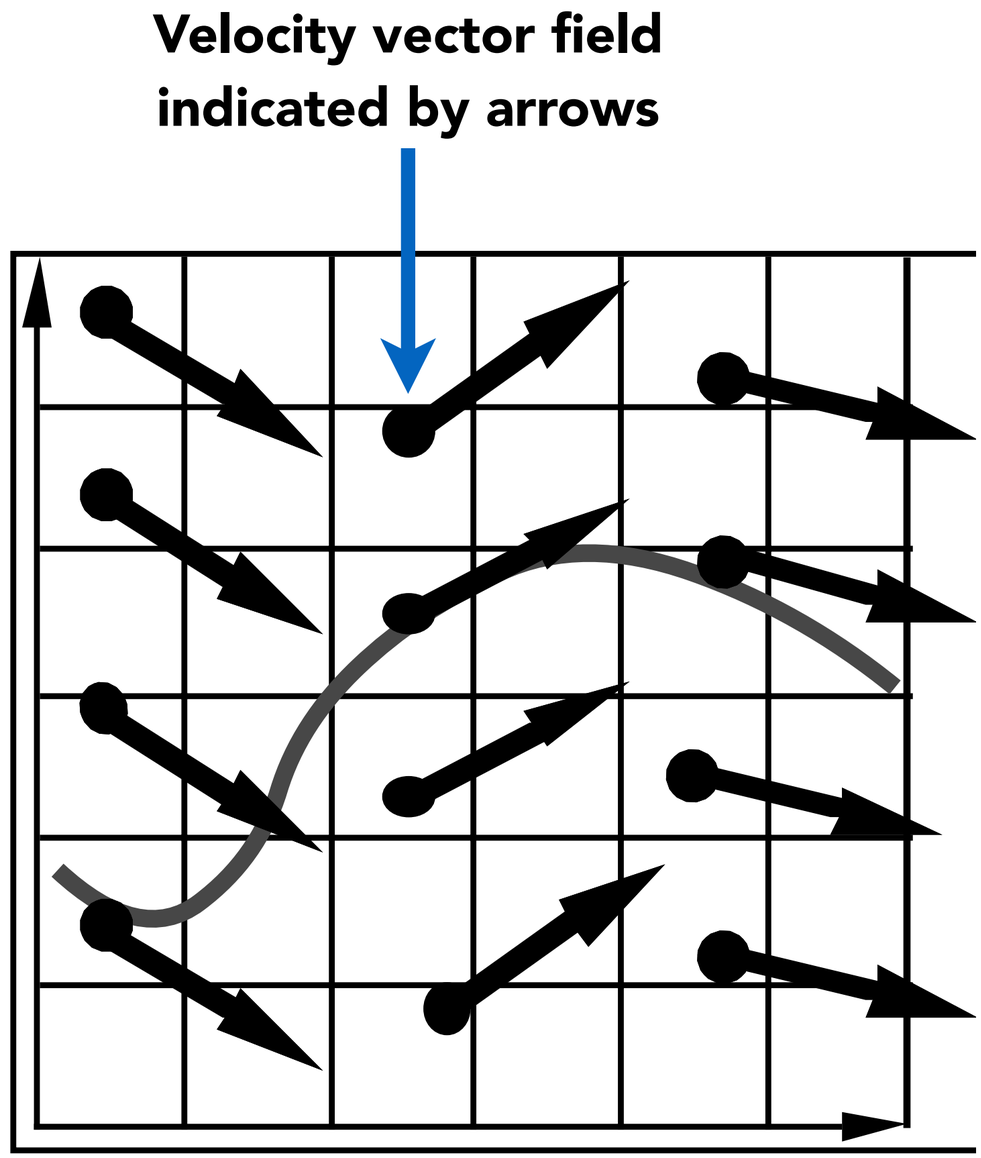
Single Particle Simulation

First study motion of a single particle

- Later, generalize to a multitude of particles

To start, assume motion of particle determined by a velocity vector field that is a function of position and time: $v(x,t)$

In example v and x are 2D



Witkin and Baraff

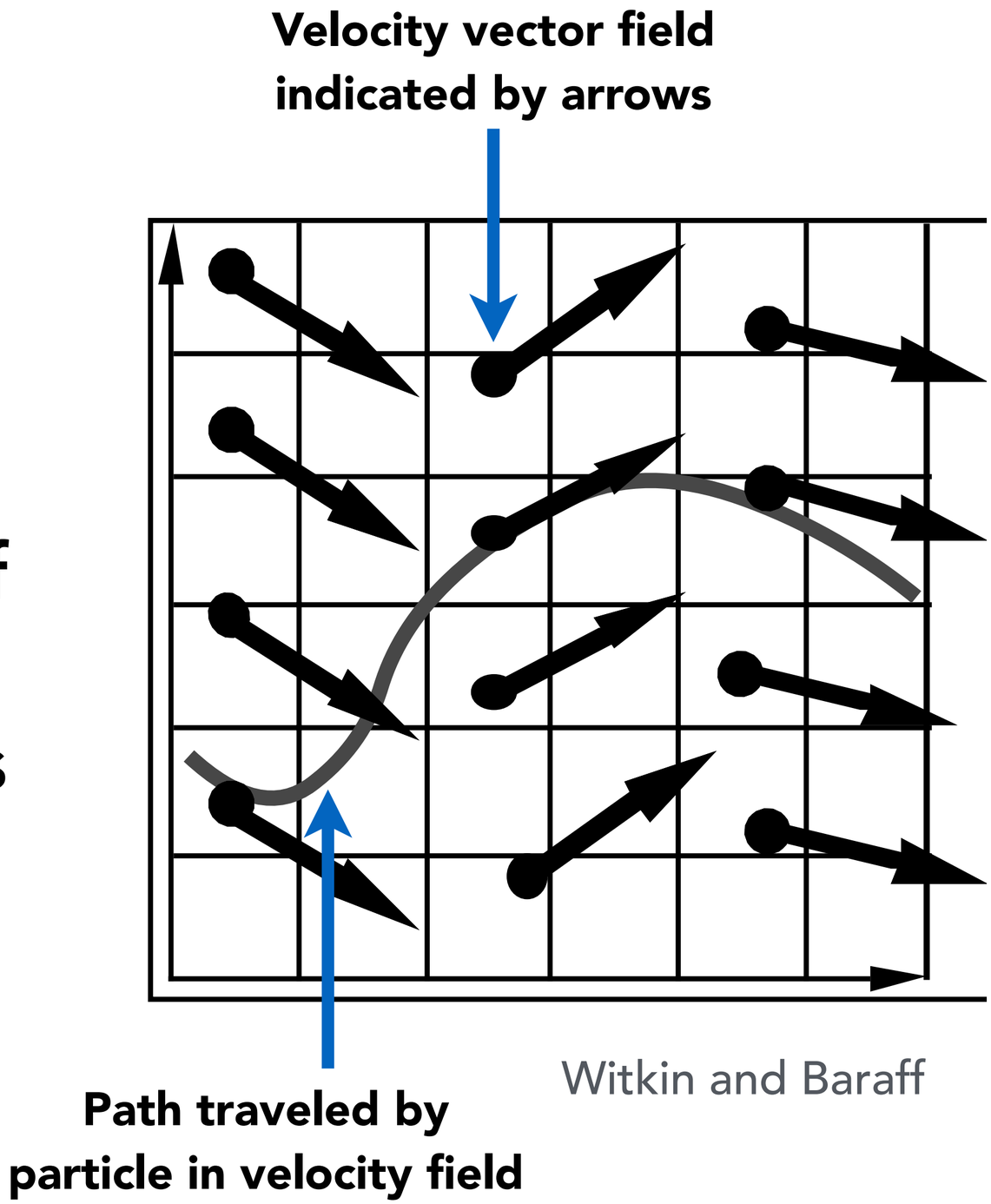
Single Particle Simulation

First study motion of a single particle

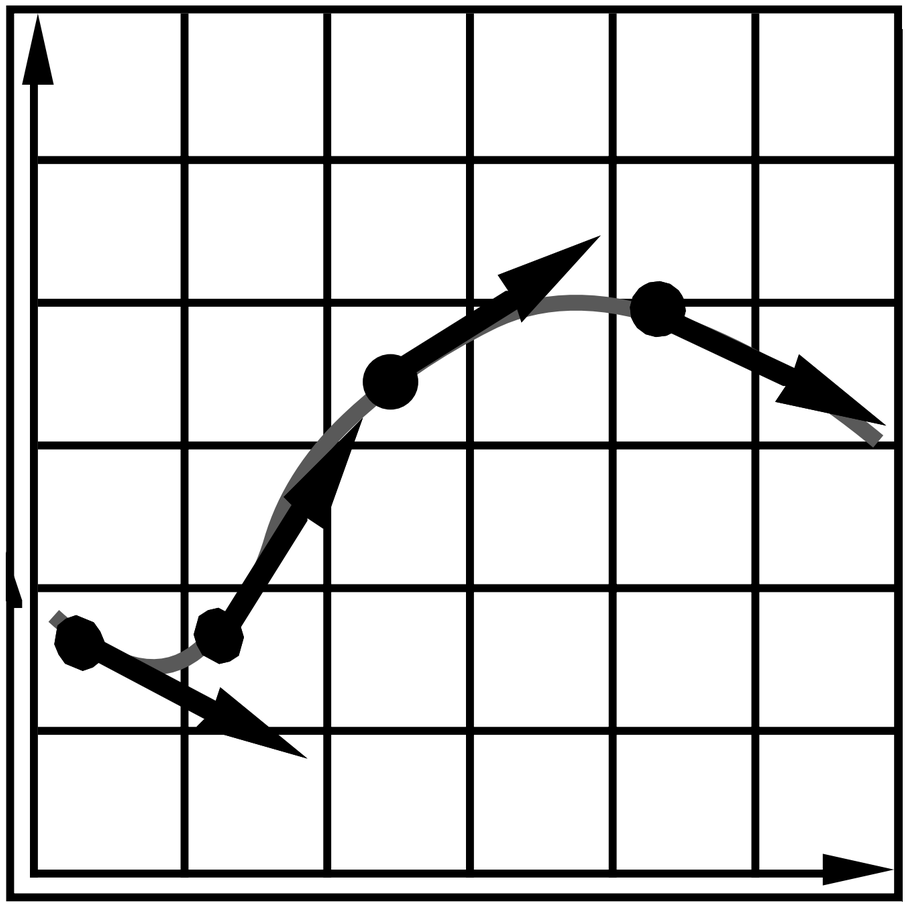
- Later, generalize to a multitude of particles

To start, assume motion of particle determined by a velocity vector field that is a function of position and time: $v(x,t)$

In example v and x are 2D



Ordinary Differential Equation (ODE)

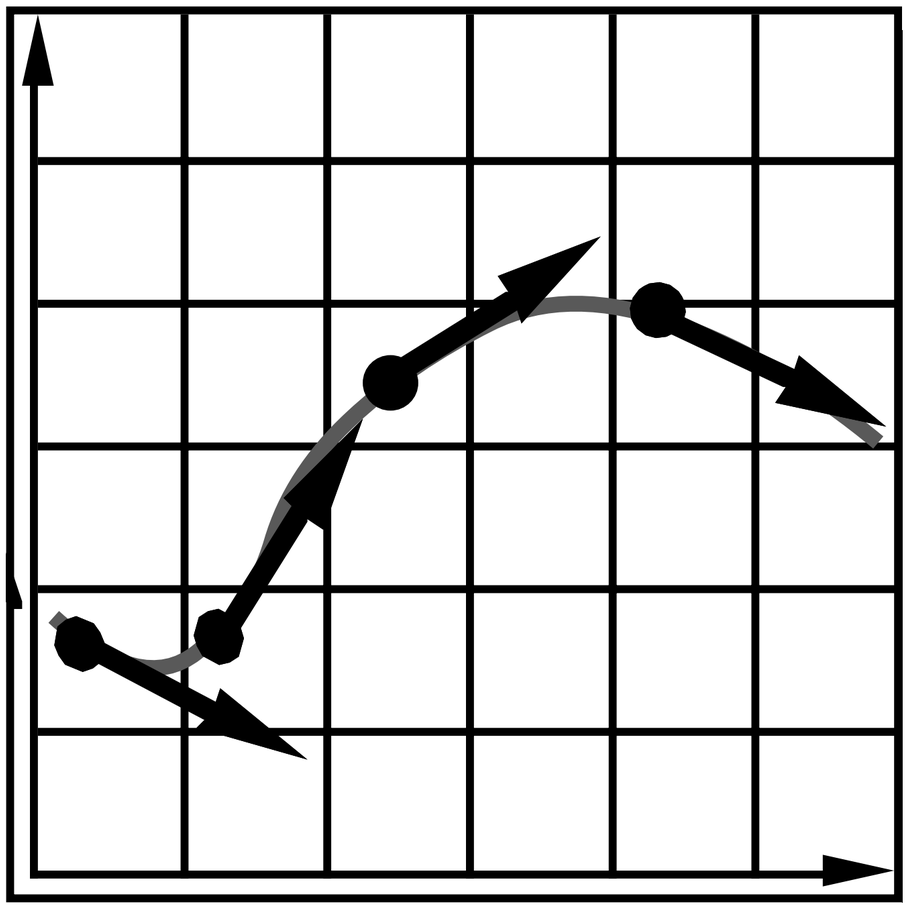


Witkin and Baraff

Ordinary Differential Equation (ODE)

Computing position of particle over time requires solving a first-order ordinary differential equation:

$$\frac{dx}{dt} = \dot{x} = v(x,t)$$



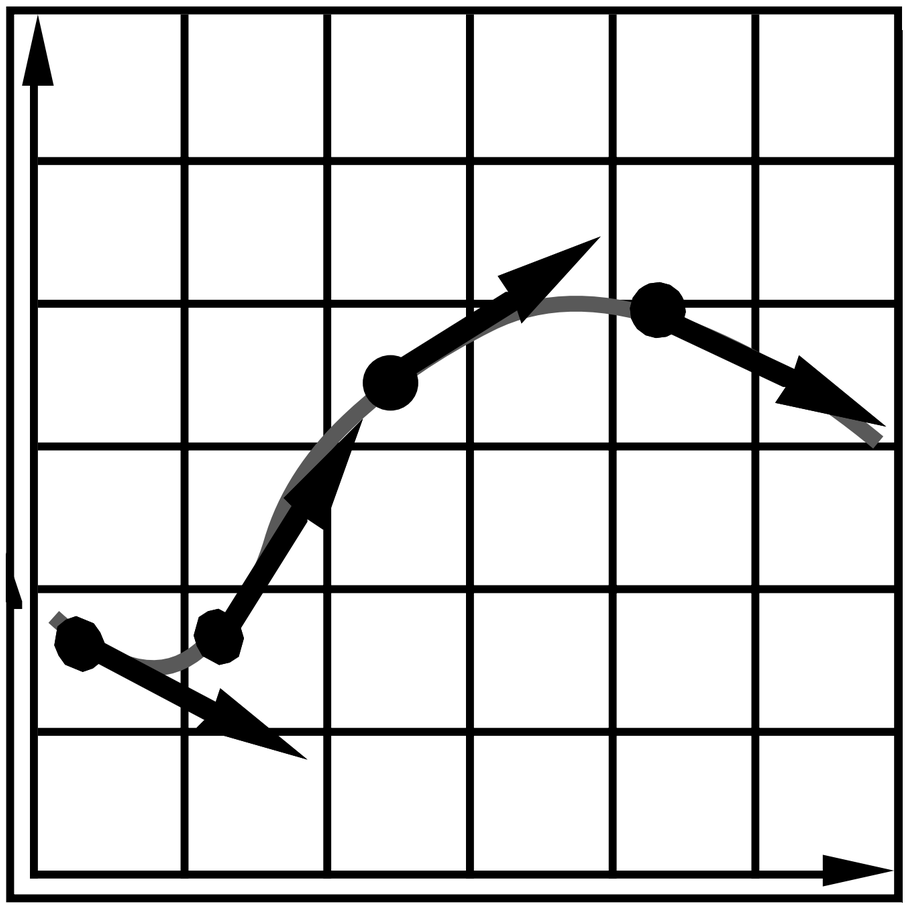
Witkin and Baraff

Ordinary Differential Equation (ODE)

Computing position of particle over time requires solving a first-order ordinary differential equation:

$$\frac{dx}{dt} = \dot{x} = v(x,t)$$

"First-order" refers to the first derivative being taken.



Witkin and Baraff

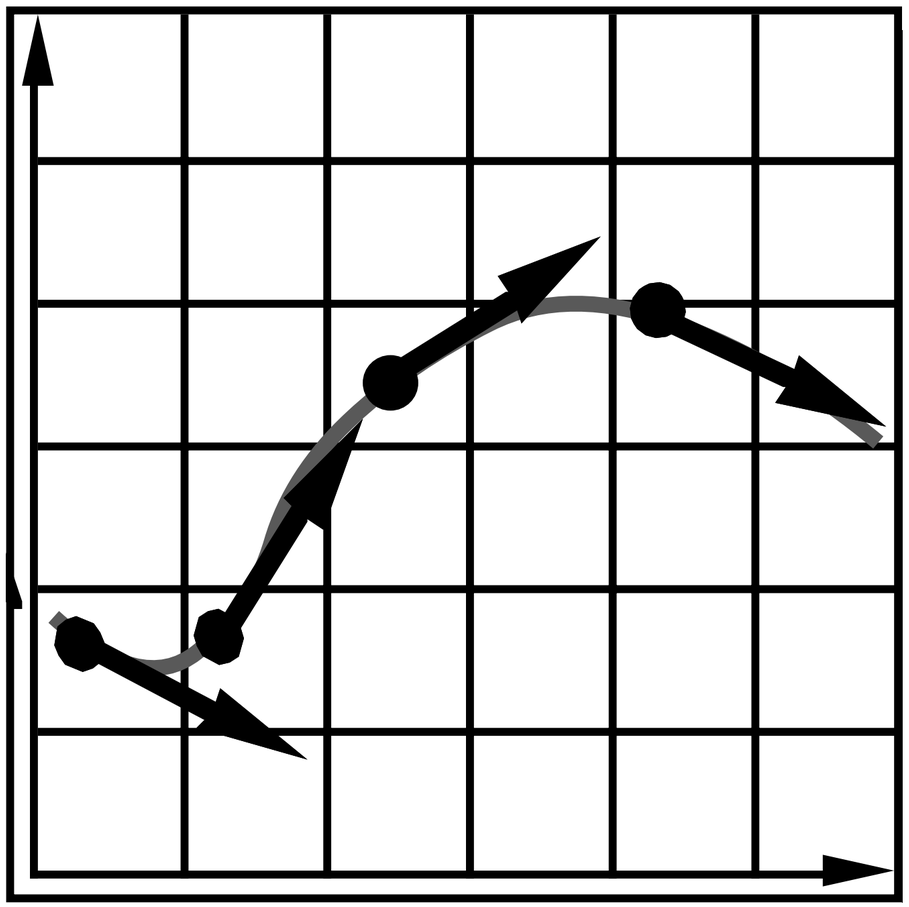
Ordinary Differential Equation (ODE)

Computing position of particle over time requires solving a first-order ordinary differential equation:

$$\frac{dx}{dt} = \dot{x} = v(x,t)$$

"First-order" refers to the first derivative being taken.

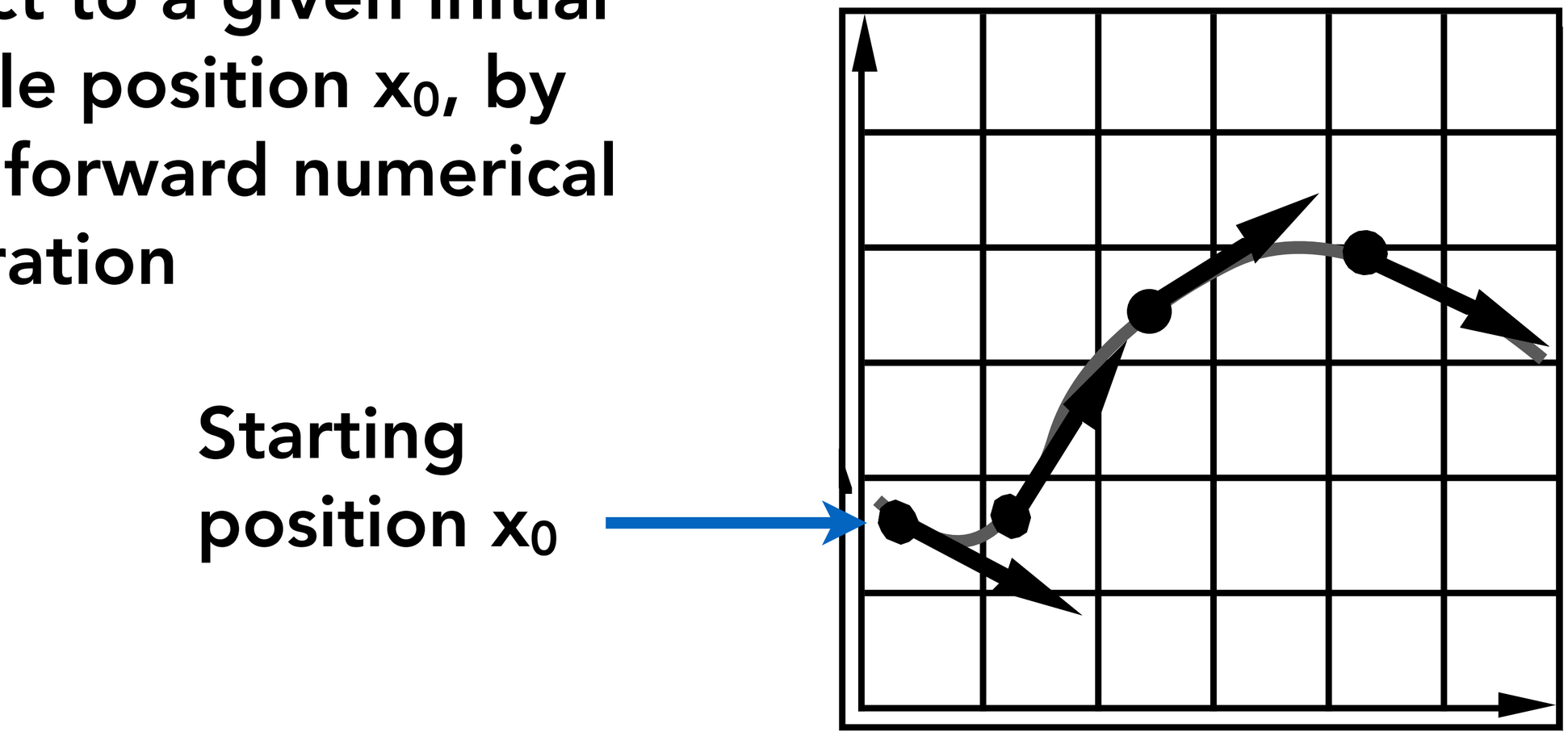
"Ordinary" refers to derivatives of time, but not derivates of space (which are "partial" differential equations)



Witkin and Baraff

Solving for Particle Position

We can solve the ODE, subject to a given initial particle position x_0, by using forward numerical integration



Witkin and Baraff

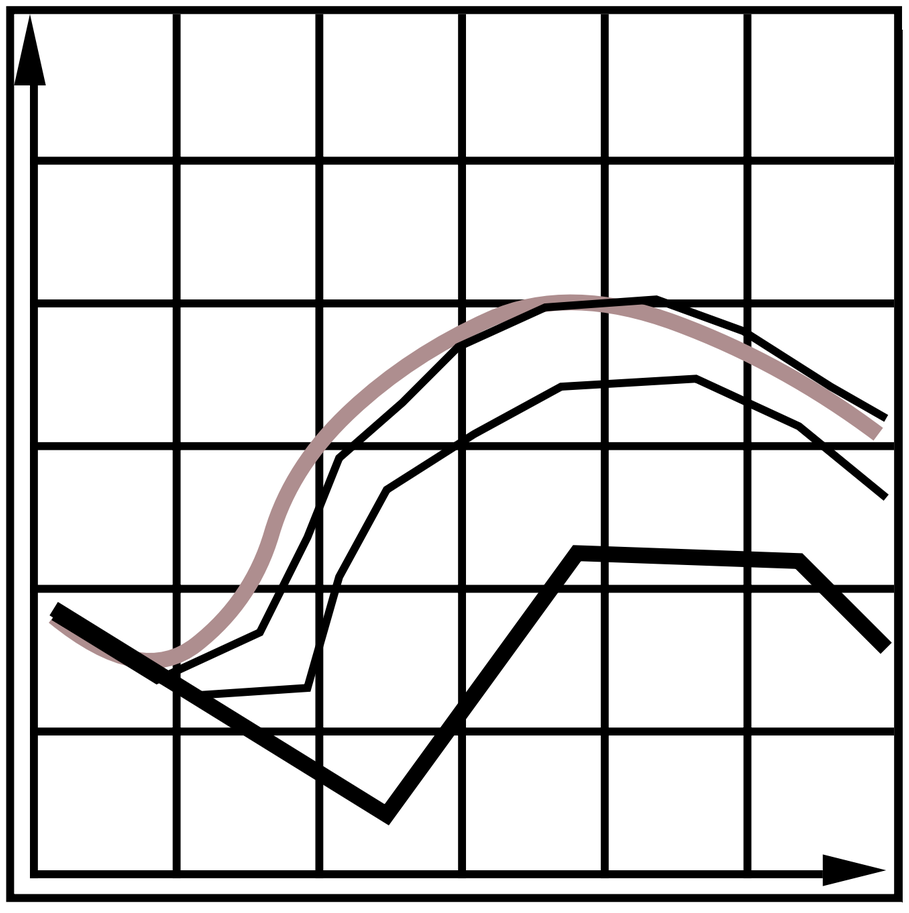
Euler's Method - Errors

With numerical integration, errors accumulate

Euler integration is particularly bad

Example:

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \Delta t\, \boldsymbol{v}(\boldsymbol{x}, t)$$



Witkin and Baraff

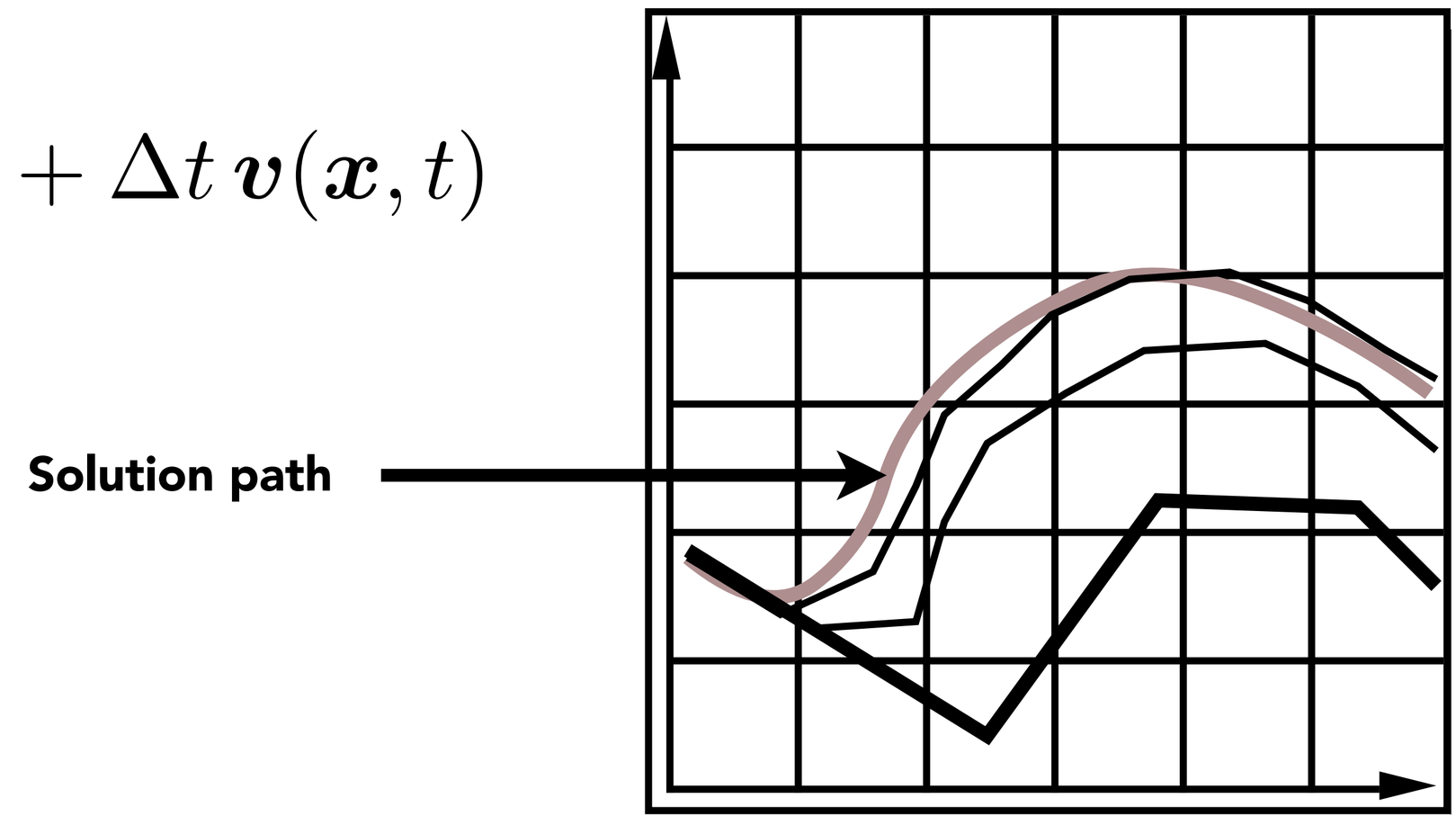
Euler's Method - Errors

With numerical integration, errors accumulate

Euler integration is particularly bad

Example:

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t\, \boldsymbol{v}(\boldsymbol{x}, t)$$



Witkin and Baraff

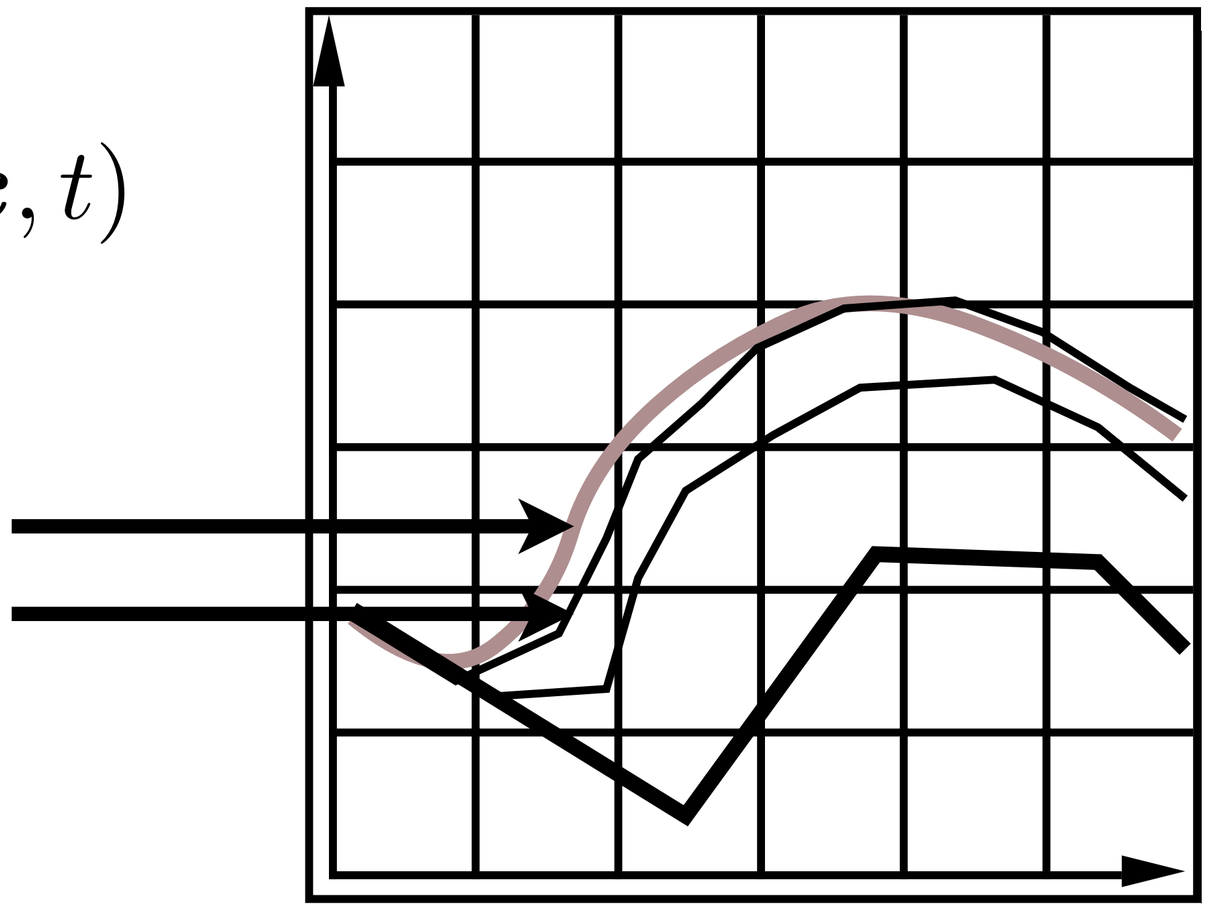
Euler's Method - Errors

With numerical integration, errors accumulate

Euler integration is particularly bad

Example:

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \Delta t\, \boldsymbol{v}(\boldsymbol{x}, t)$$



Witkin and Baraff

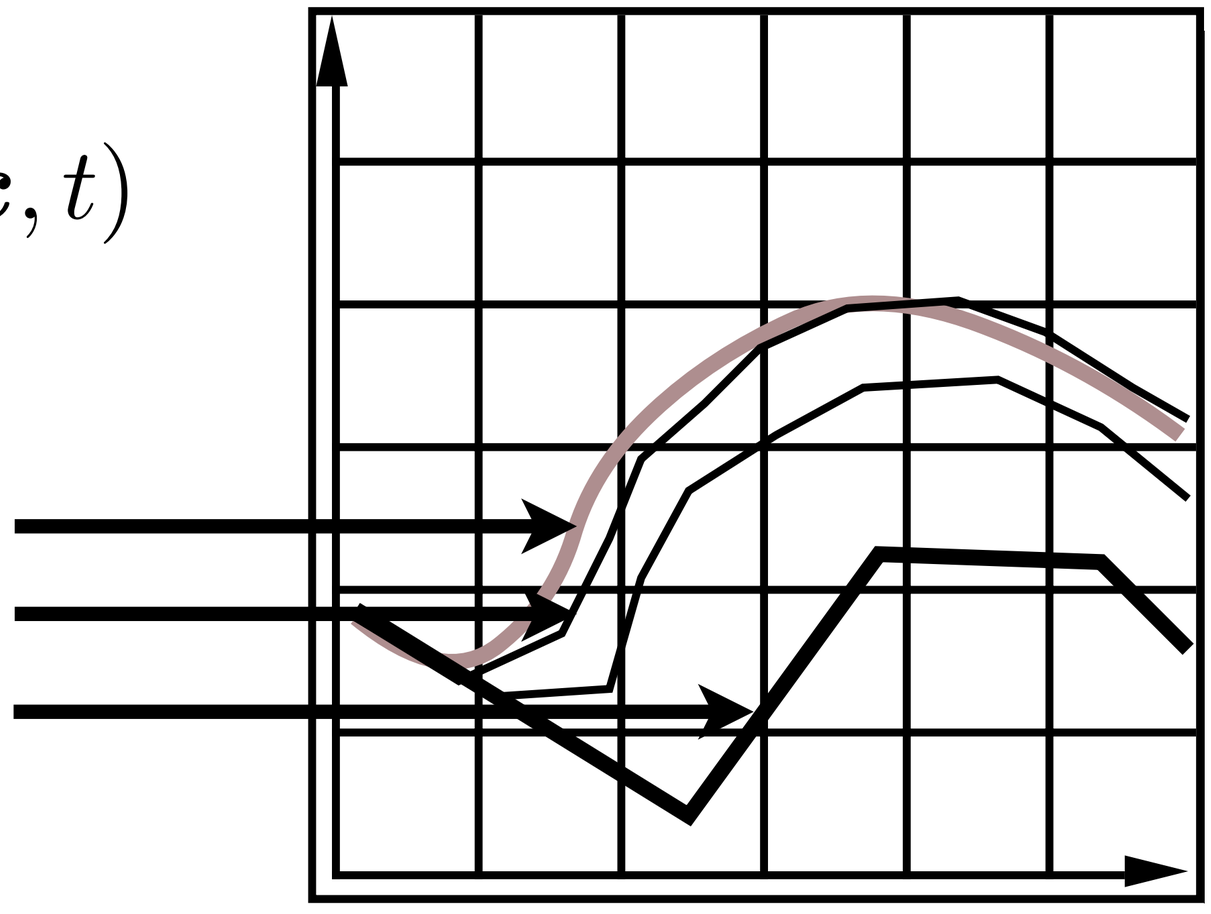
Euler's Method - Errors

With numerical integration, errors accumulate

Euler integration is particularly bad

Example:

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \Delta t\, \boldsymbol{v}(\boldsymbol{x}, t)$$



Witkin and Baraff

Errors and Instability

Errors and Instability

Solving by numerical integration with finite differences leads to two problems

Errors and Instability

Solving by numerical integration with finite differences leads to two problems

Errors

- Errors at each time step accumulate. Accuracy decreases as simulation proceeds
- Accuracy may not be critical in graphics applications

Errors and Instability

Solving by numerical integration with finite differences leads to two problems

Errors

- Errors at each time step accumulate. Accuracy decreases as simulation proceeds
- Accuracy may not be critical in graphics applications

Instability

- Errors can compound, causing the simulation to diverge even when the underlying system does not
- Lack of stability is a fundamental problem in simulation, and cannot be ignored

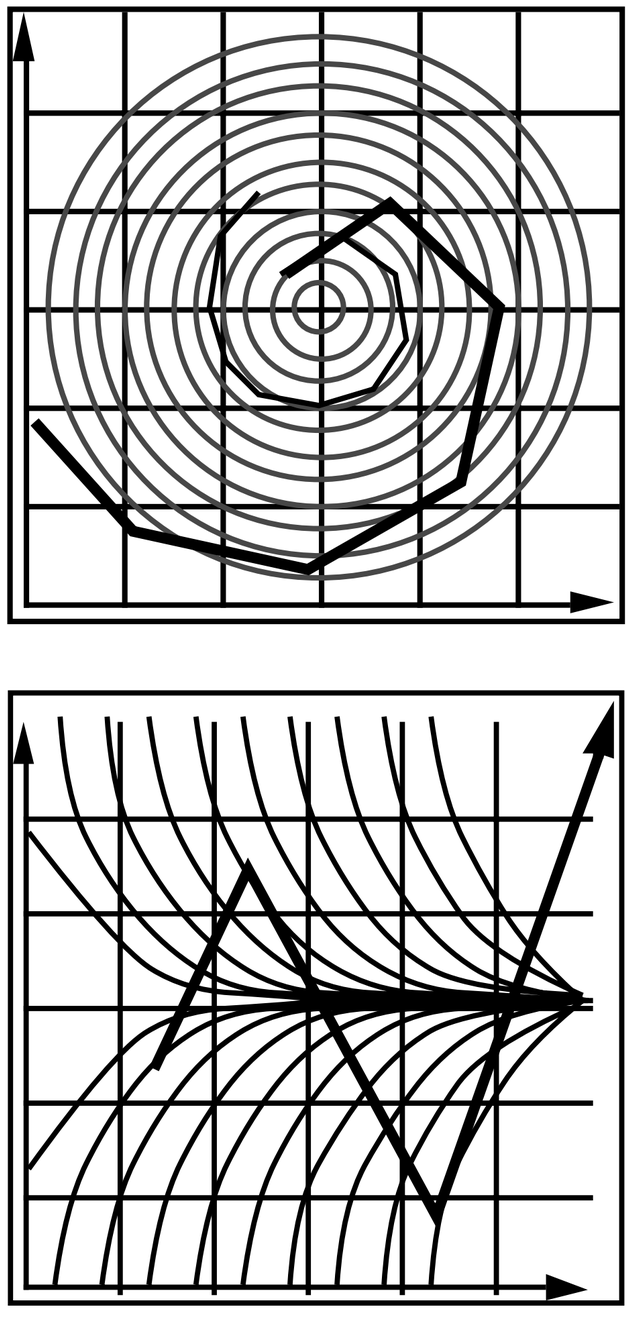
Instability of Forward Euler Method

Forward Euler (explicit)

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t\, \boldsymbol{v}(\boldsymbol{x}, t)$$

Two key problems:

- Inaccuracies increase as time step Δt increases
- Instability is a common, serious problem that can cause simulation to diverge



Witkin and Baraff

Instability Example (Spring)



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

Instability Example (Spring)



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

When mass is moving inward:

- Force is decreasing
- Each time-step overestimates the velocity change (increases energy)

Instability Example (Spring)



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

When mass is moving inward:

- **Force is decreasing**
- **Each time-step overestimates the velocity change (increases energy)**

When mass gets to origin

- **Has velocity that is too high, now traveling outward**

Instability Example (Spring)



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

When mass is moving inward:

- **Force is decreasing**
- **Each time-step overestimates the velocity change (increases energy)**

When mass gets to origin

- **Has velocity that is too high, now traveling outward**

When mass is moving outward

- **Force is increasing**
- **Each time-step underestimates the velocity change (increases energy)**

Instability Example (Spring)



$$\boldsymbol{f}_{a \to b} = k_s(\boldsymbol{b} - \boldsymbol{a})$$

When mass is moving inward:

- **Force is decreasing**
- **Each time-step overestimates the velocity change (increases energy)**

When mass gets to origin

- **Has velocity that is too high, now traveling outward**

When mass is moving outward

- **Force is increasing**
- **Each time-step underestimates the velocity change (increases energy)**

At each motion cycle, mass gains energy exponentially

Some Methods to Combat Instability

Modified Euler

- Average velocities at start and endpoint

Adaptive step size

- Compare one step and two half-steps, recursively, until error is acceptable

Implicit methods

- Use the velocity at the next time step (hard)

Position-based / Verlet integration

- Constrain positions and velocities of particles after time step

Modified Euler

Modified Euler

- Average velocity at start and end of step
- OK if system is not very stiff (k_s small enough)
- But, still unstable

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \frac{\Delta t}{2}\left(\dot{\boldsymbol{x}}^t + \dot{\boldsymbol{x}}^{t+\Delta t}\right)$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \dot{\boldsymbol{x}}^t + \Delta t\, \ddot{\boldsymbol{x}}^t$$

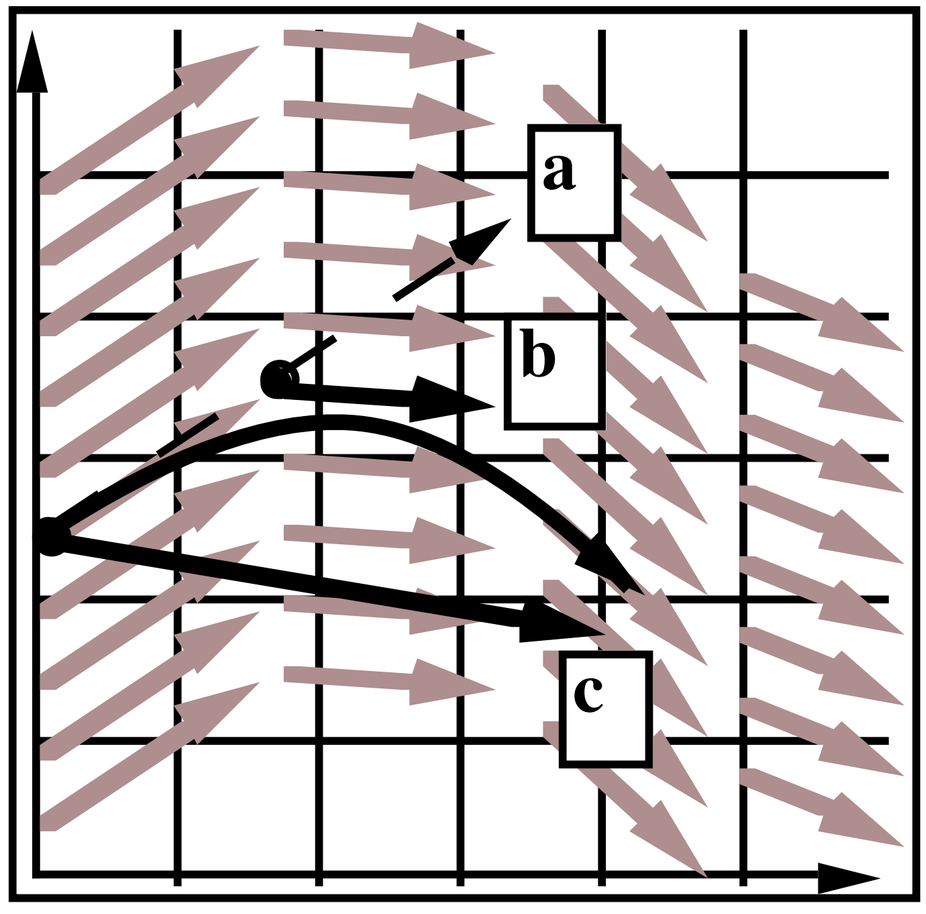
$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^t + \Delta t\, \dot{\boldsymbol{x}}^t + \frac{(\Delta t)^2}{2}\, \ddot{\boldsymbol{x}}^t$$

Midpoint Method

Midpoint method

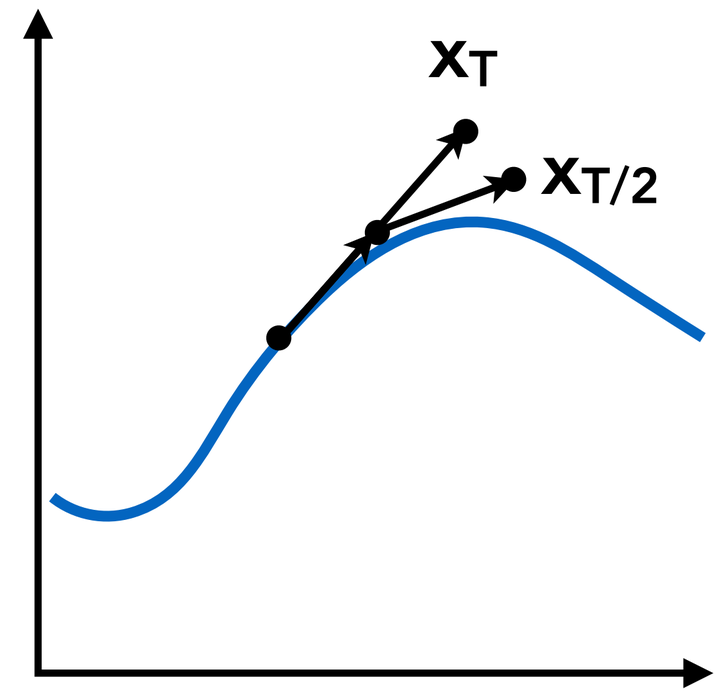
- Compute Euler step (a)
- Compute derivative at midpoint of Euler step (b)
- Update position using midpoint derivative (c)

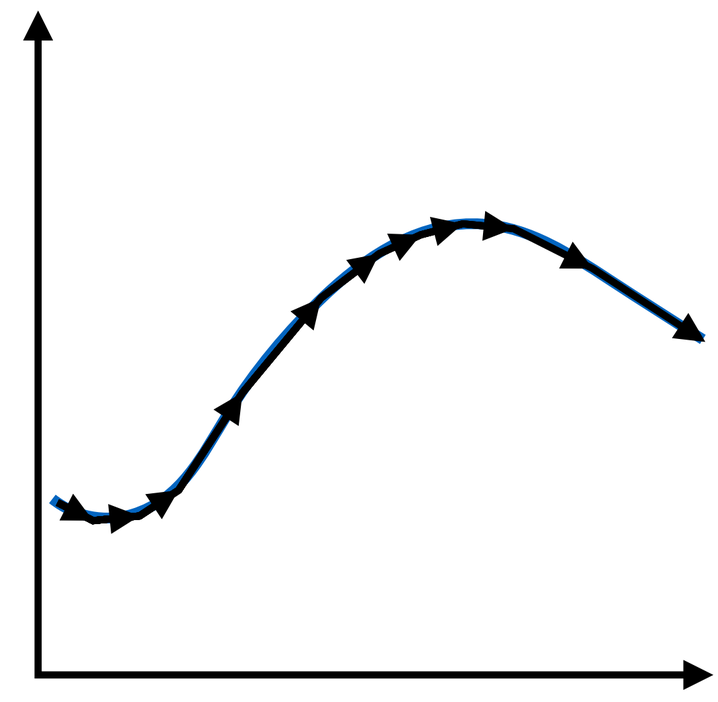
$$x_{\text{mid}} = x(t) + \Delta t/2 \cdot v(x(t), t)$$
$$x(t + \Delta t) = x(t) + \Delta t \cdot v(x_{\text{mid}}, t)$$



Witkin and Baraff

Adaptive Step Size



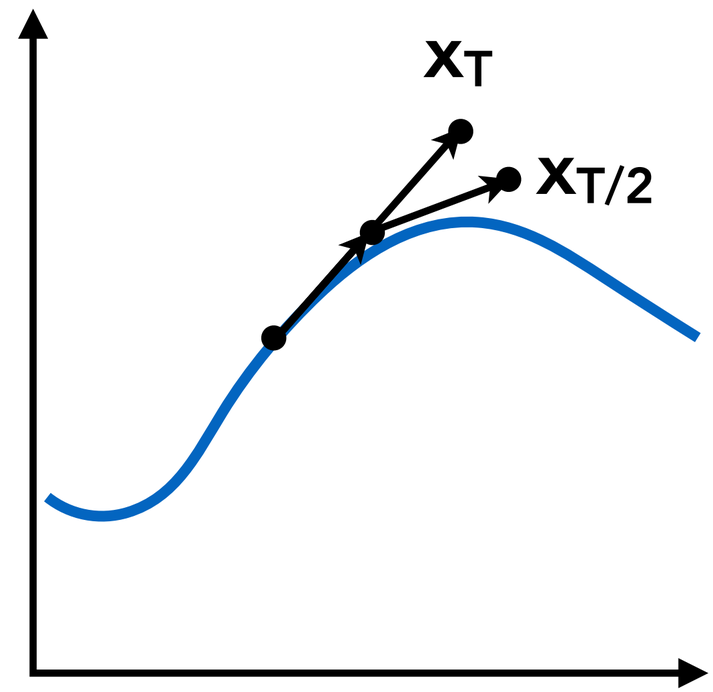


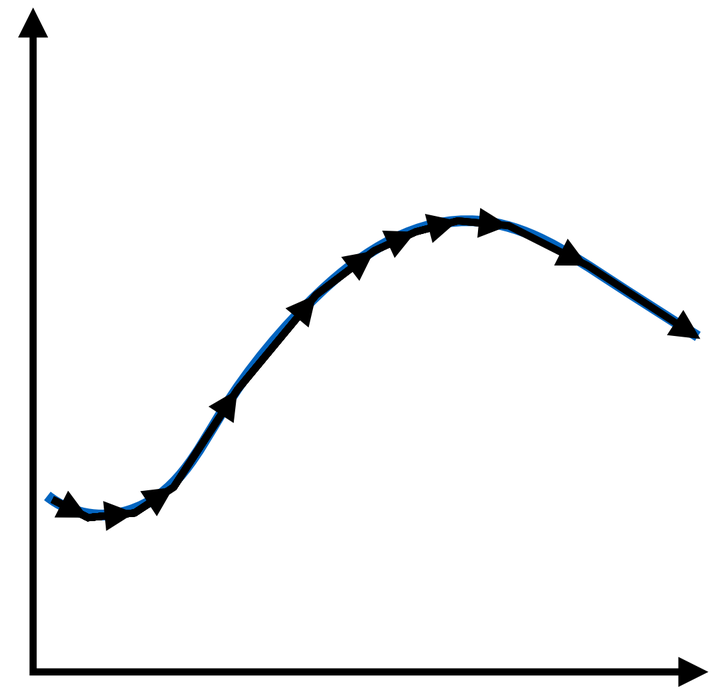
Slide credit: Funkhouser

Adaptive Step Size

Adaptive step size

- Technique for choosing step size based on error estimate
- Highly recommended technique
- But may need very small steps!





Slide credit: Funkhouser

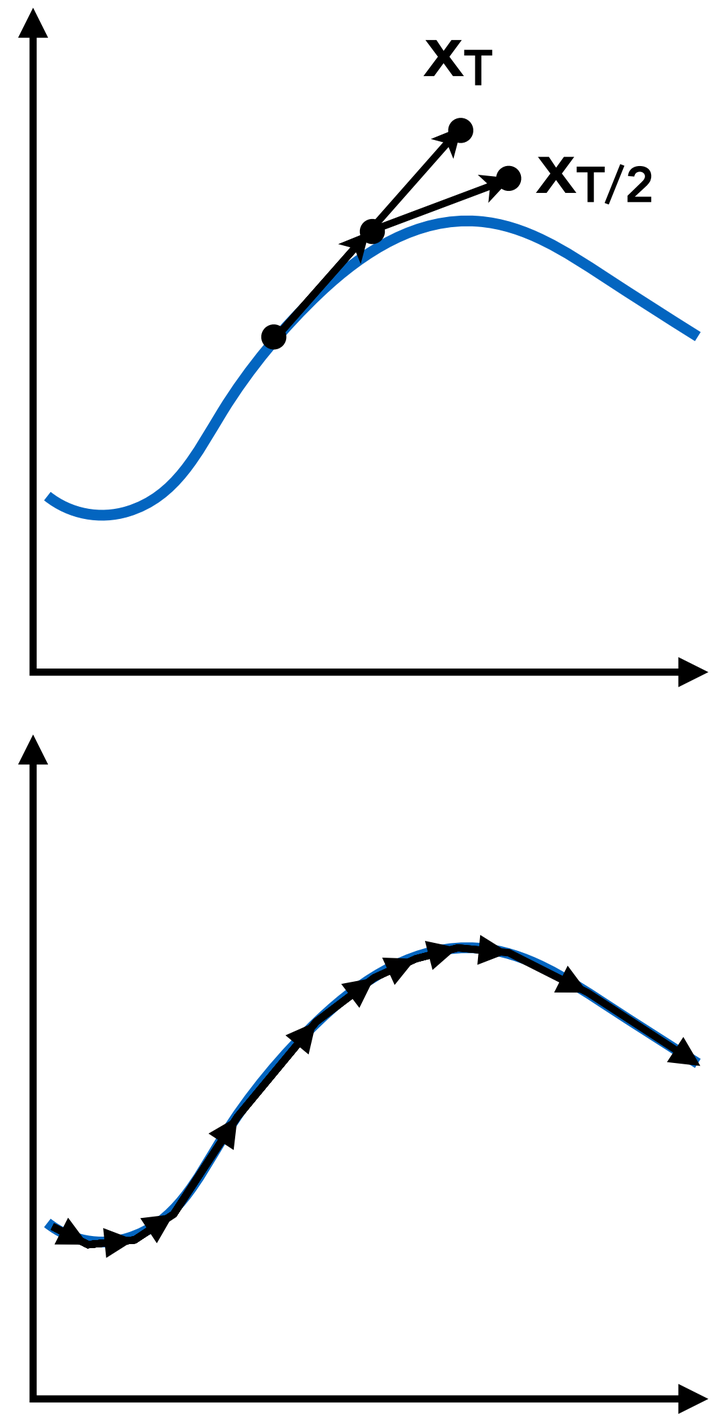
Adaptive Step Size

Adaptive step size

- Technique for choosing step size based on error estimate
- Highly recommended technique
- But may need very small steps!

Repeat until error is below threshold:

- Compute x_T an Euler step, size T
- Compute $x_{T/2}$ two Euler steps, size T/2
- Compute error $\| x_T - x_{T/2} \|$
- If (error > threshold) reduce step size and try again



Slide credit: Funkhouser

Implicit Euler Method

Implicit methods

- Informally called backward methods
- Use derivatives in the future, for the current step

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t\, \dot{\boldsymbol{x}}^{t+\Delta t}$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \dot{\boldsymbol{x}}^{t} + \Delta t\, \ddot{\boldsymbol{x}}^{t+\Delta t}$$

Implicit Euler Method

Implicit methods

- Informally called backward methods
- Use derivatives in the future, for the current step

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t\, \dot{\boldsymbol{x}}^{t+\Delta t}$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \dot{\boldsymbol{x}}^{t} + \Delta t\, \ddot{\boldsymbol{x}}^{t+\Delta t}$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \mathsf{V}(\boldsymbol{x}^{t+\Delta t}, \dot{\boldsymbol{x}}^{t+\Delta t}, t + \Delta t)$$

$$\ddot{\boldsymbol{x}}^{t+\Delta t} = \mathsf{A}(\boldsymbol{x}^{t+\Delta t}, \dot{\boldsymbol{x}}^{t+\Delta t}, t + \Delta t)$$

Implicit Euler Method

Implicit methods

- Informally called backward methods
- Use derivatives in the future, for the current step

$$\boldsymbol{x}^{t+\Delta t} = \boldsymbol{x}^{t} + \Delta t \, \mathsf{V}(\boldsymbol{x}^{t+\Delta t}, \dot{\boldsymbol{x}}^{t+\Delta t}, t + \Delta t)$$

$$\dot{\boldsymbol{x}}^{t+\Delta t} = \dot{\boldsymbol{x}}^{t} + \Delta t \, \mathsf{A}(\boldsymbol{x}^{t+\Delta t}, \dot{\boldsymbol{x}}^{t+\Delta t}, t + \Delta t)$$

- Solve nonlinear problem for $x^{t+\Delta t}$ and $\dot{x}^{t+\Delta t}$
- Use root-finding algorithm, e.g. Newton's method
- Can be made unconditionally stable

Position-Based / Verlet Integration

Idea:

- After modified Euler forward-step, constrain positions of particles to prevent divergent, unstable behavior
- Use constrained positions to calculate velocity
- Both of these ideas will dissipate energy, stabilize

Pros / cons

- Fast and simple
- Not physically based, dissipates energy (error)
- Highly recommended (assignment)

Position-Based / Verlet Integration

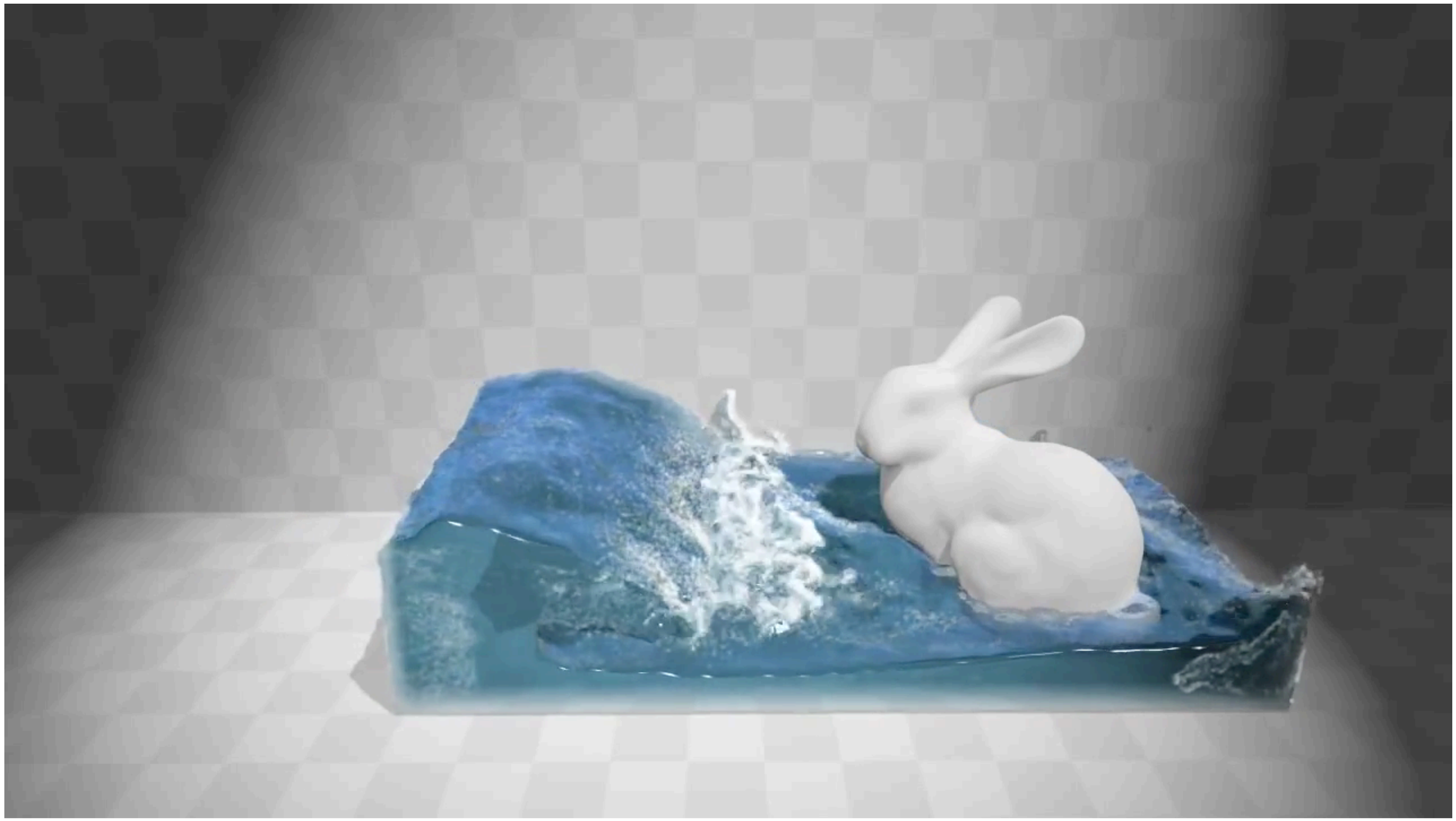
Algorithm 1 Position-based dynamics

for all vertices i **do**
  initialize $\mathbf{x}_i = \mathbf{x}_i^0$, $\mathbf{v}_i = \mathbf{v}_i^0$, $w_i = 1/m_i$
end for
loop
  for all vertices i **do** $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{\text{ext}}(\mathbf{x}_i)$
  for all vertices i **do** $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$
  for all vertices i **do** genCollConstraints($\mathbf{x}_i \rightarrow \mathbf{p}_i$)
  loop solverIteration **times**
    projectConstraints($C_1, \ldots, C_{M+M_{\text{Coll}}}, \mathbf{p}_1, \ldots, \mathbf{p}_N$)
  end loop
  for all vertices i **do**
    $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i)/\Delta t$
    $\mathbf{x}_i \leftarrow \mathbf{p}_i$
  end for
  velocityUpdate($\mathbf{v}_1, \ldots, \mathbf{v}_N$)
end loop

Position-Based Simulation Methods in Computer Graphics
Bender, Müller, Macklin, Eurographics 2015

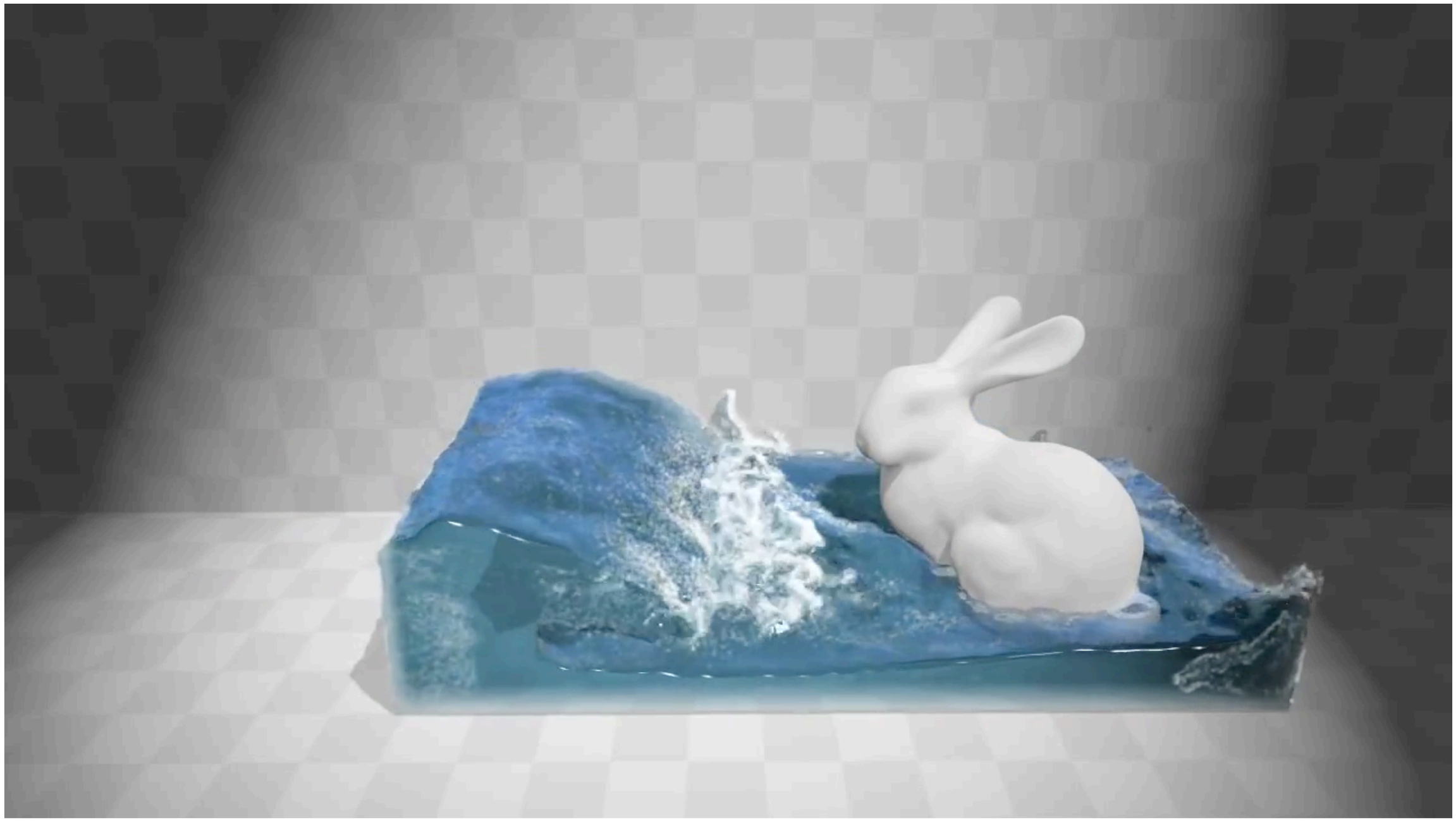
Other Particle Systems

Example: Particle-Based Fluids



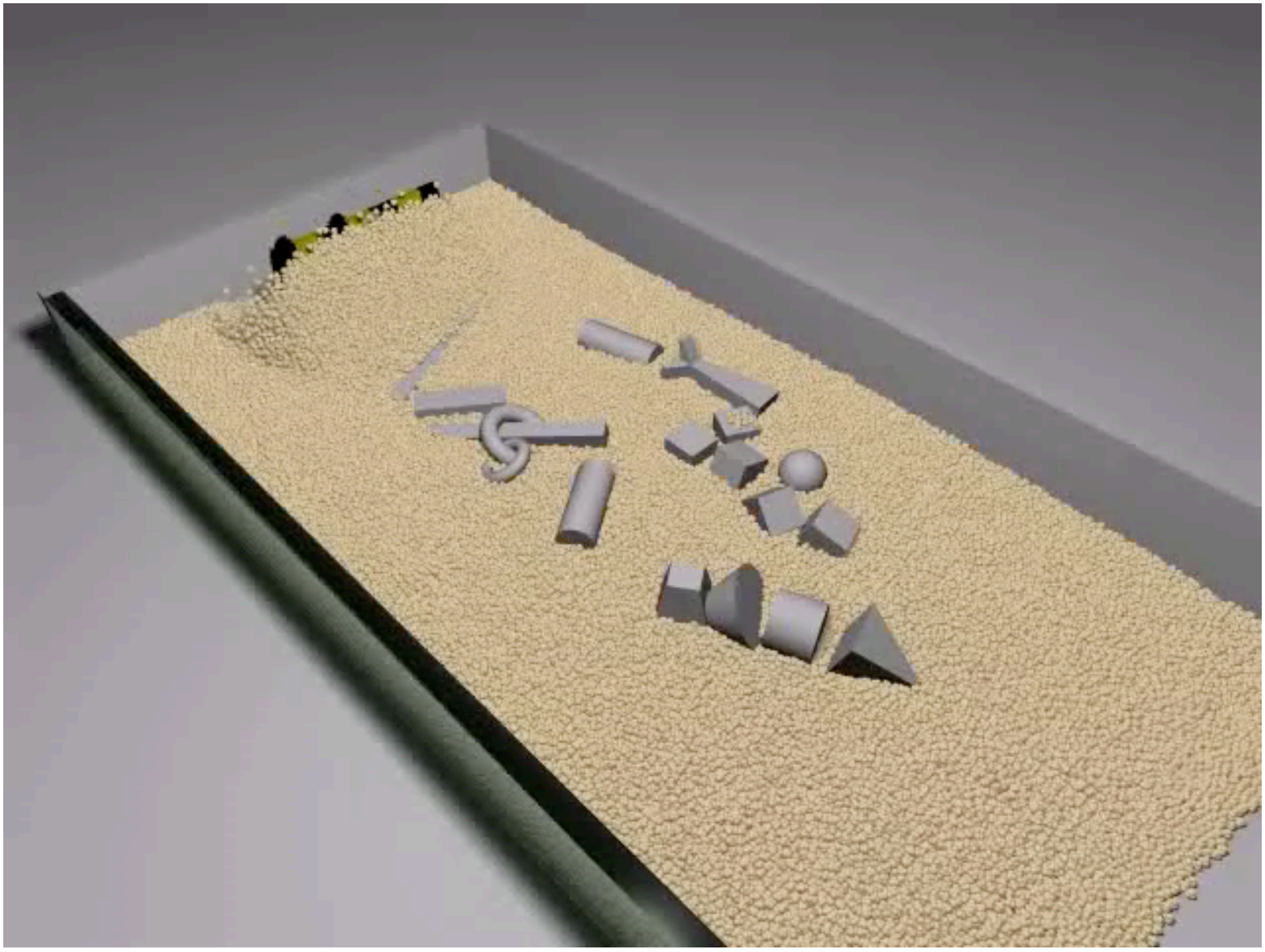
Macklin and Müller, Position Based Fluids , TOG 2013

Example: Particle-Based Fluids



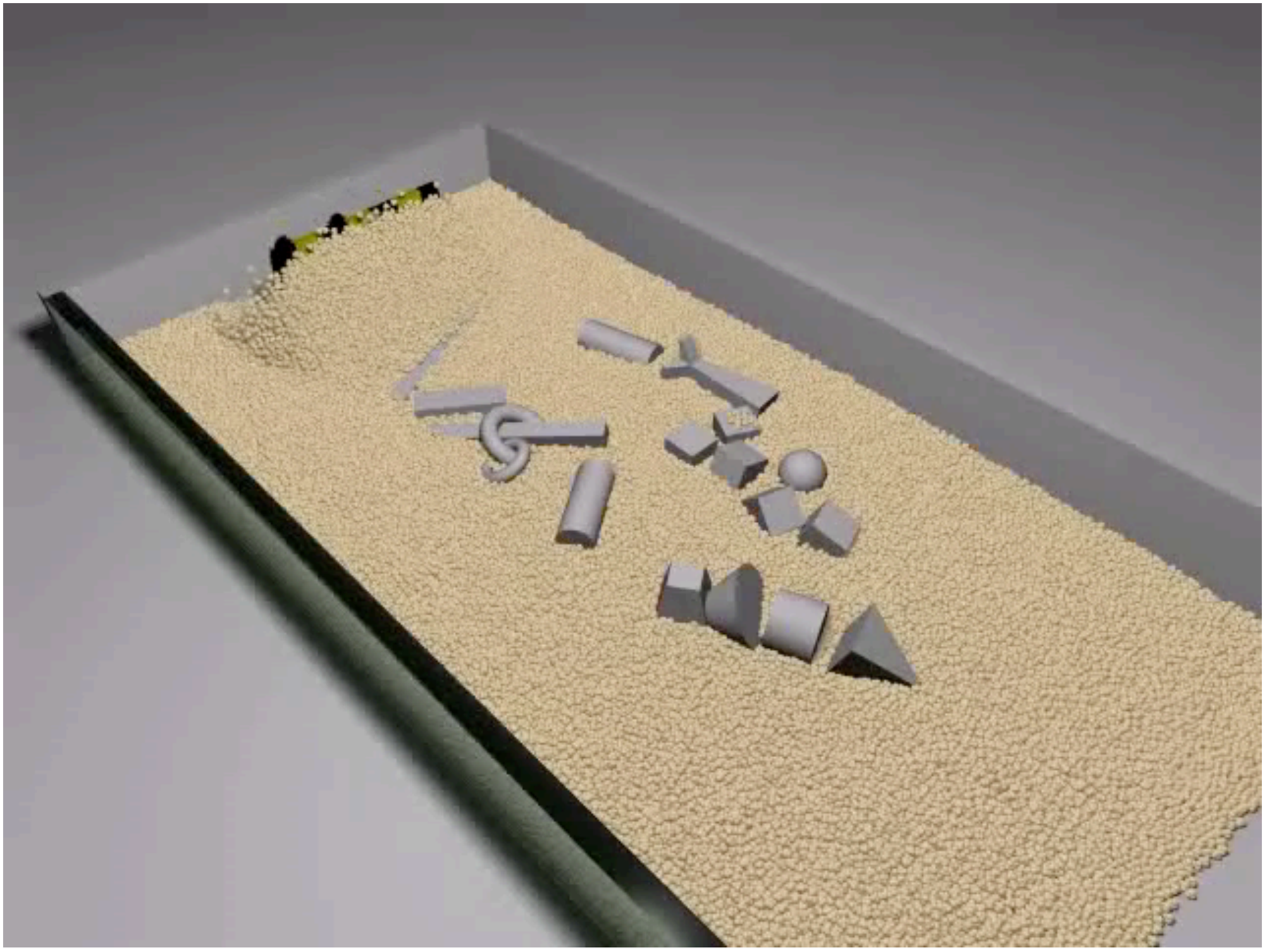
Macklin and Müller, Position Based Fluids , TOG 2013

Example: Granular Materials



Bell et al, “Particle-Based Simulation of Granular Materials”

Example: Granular Materials



Bell et al, “Particle-Based Simulation of Granular Materials”

Example: Flocking Birds



Example: Flocking Birds



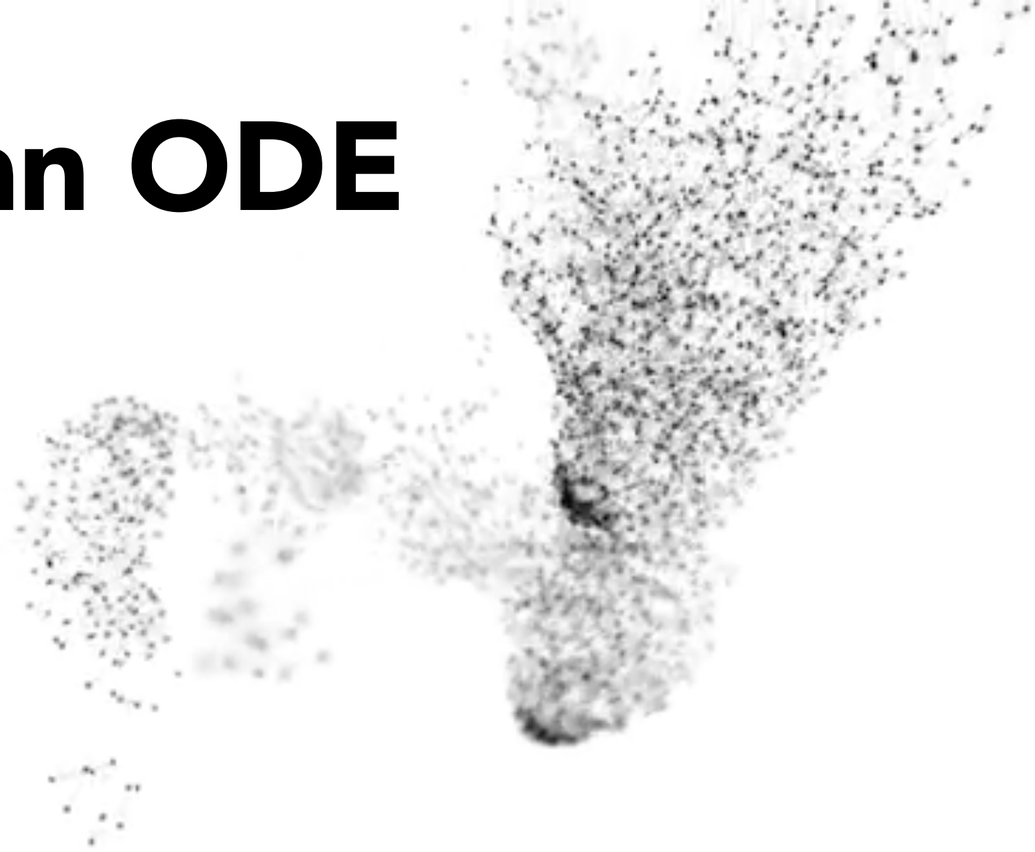
Simulated Flocking as an ODE

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

Simulated Flocking as an ODE

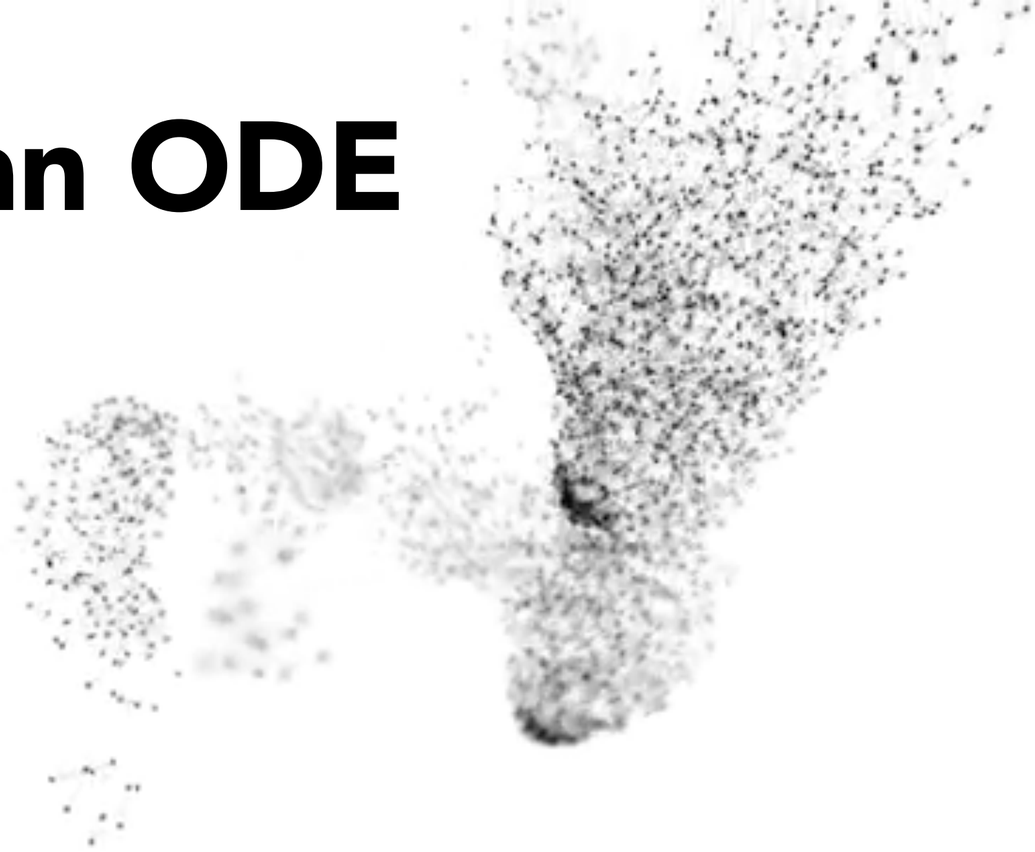
Model each bird as a particle



Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

Simulated Flocking as an ODE



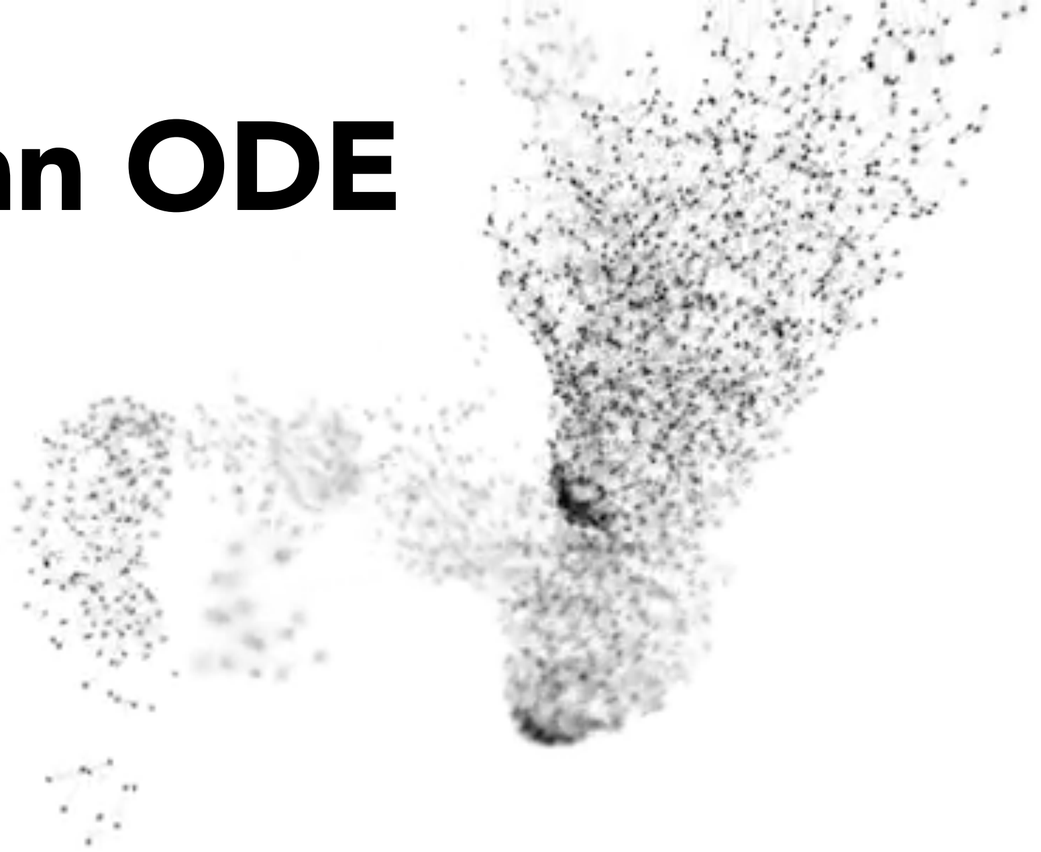
Model each bird as a particle

Subject to very simple forces:

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

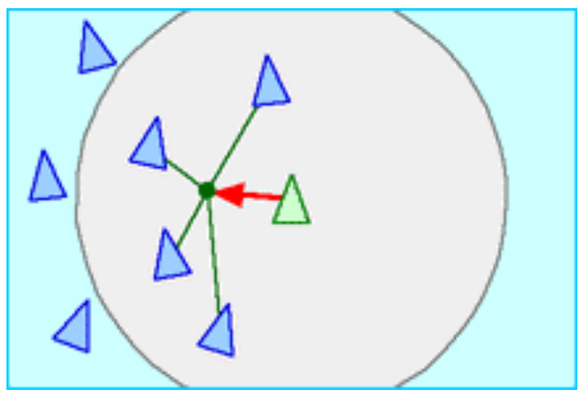
Simulated Flocking as an ODE



Model each bird as a particle

Subject to very simple forces:

- attraction to center of neighbors

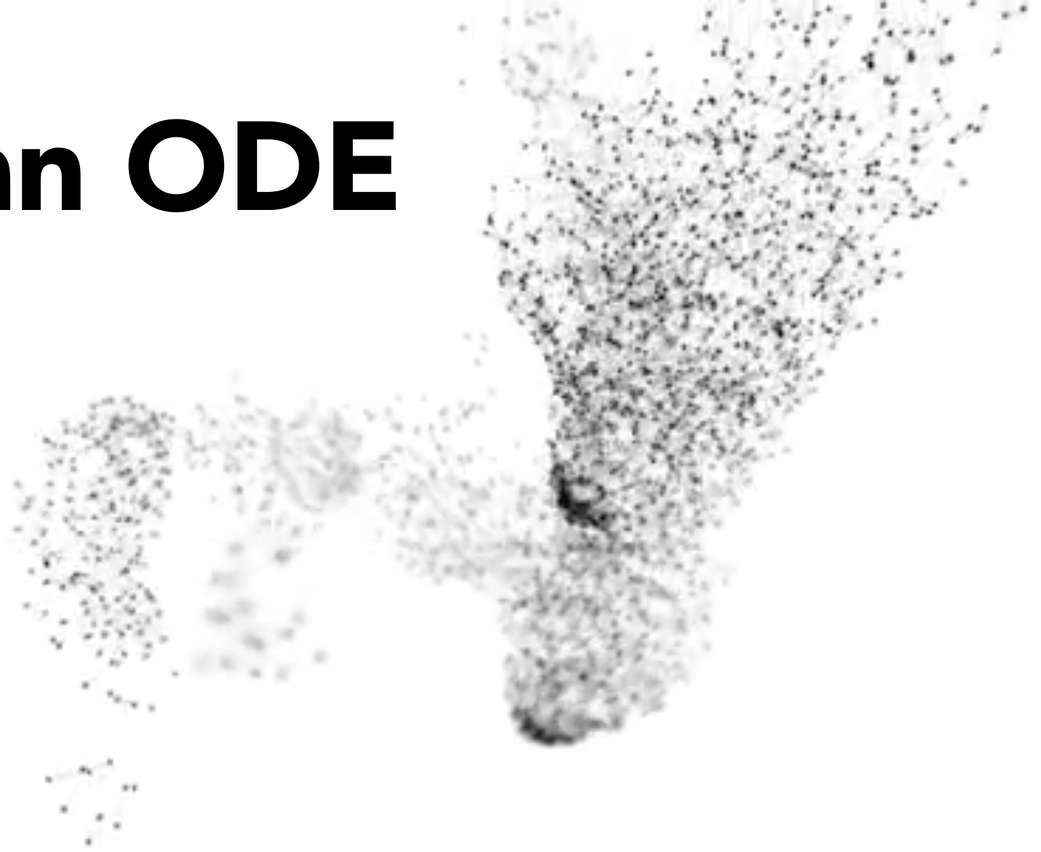


attraction

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

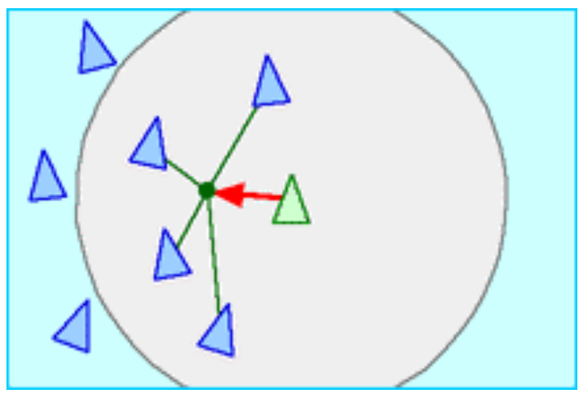
Simulated Flocking as an ODE



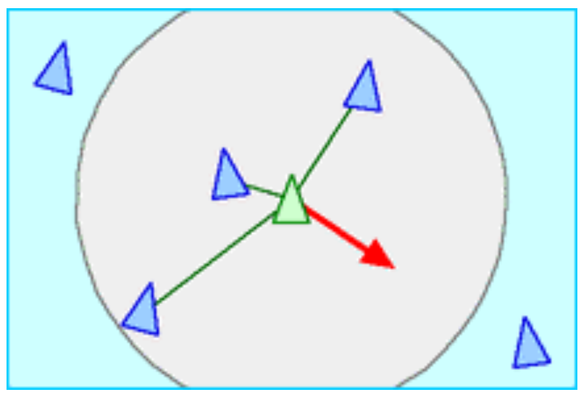
Model each bird as a particle

Subject to very simple forces:

- attraction to center of neighbors
- repulsion from individual neighbors



attraction

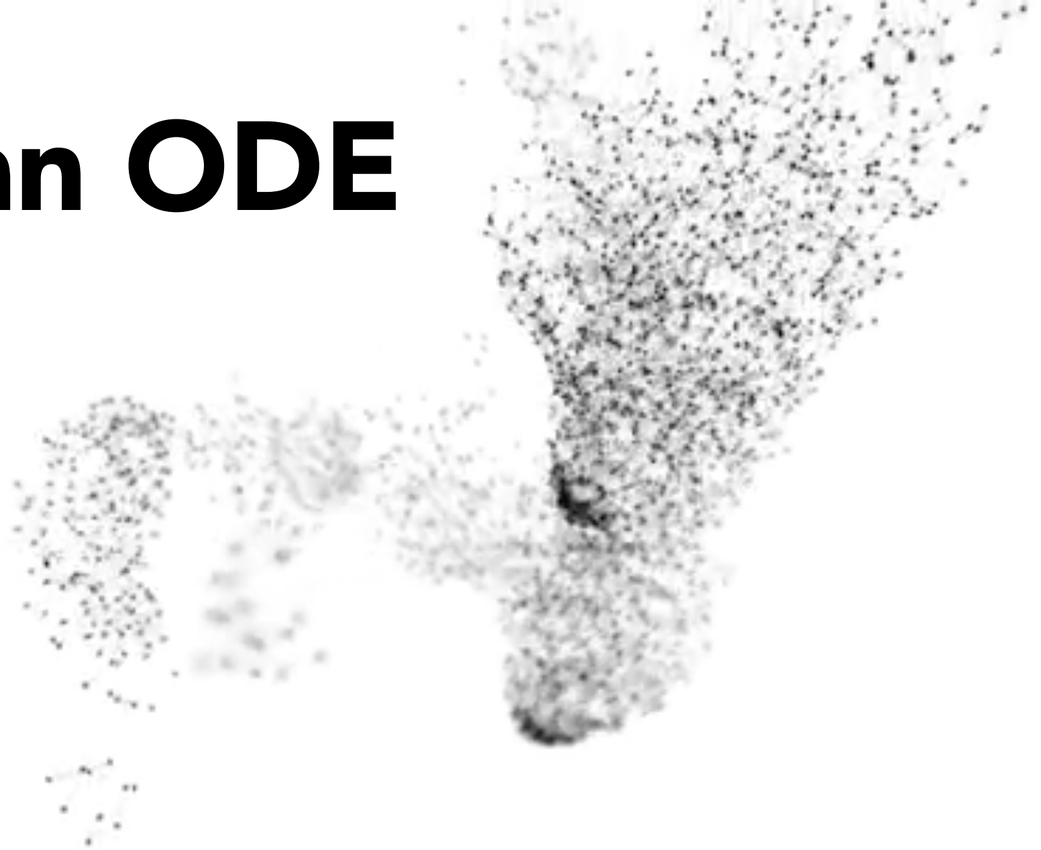


repulsion

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

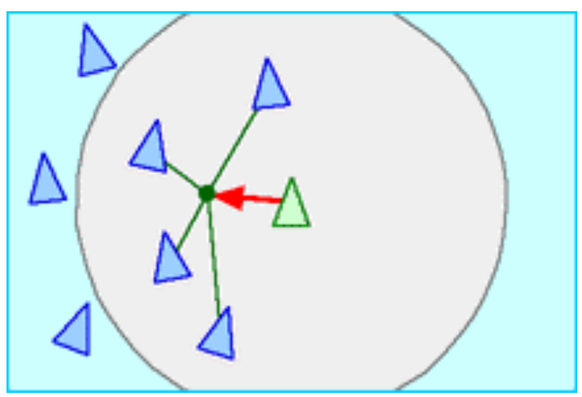
Simulated Flocking as an ODE



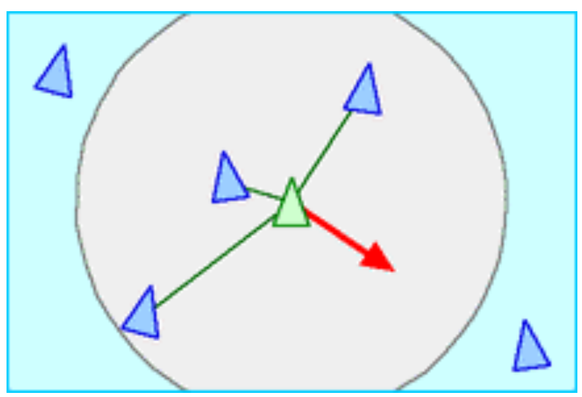
Model each bird as a particle

Subject to very simple forces:

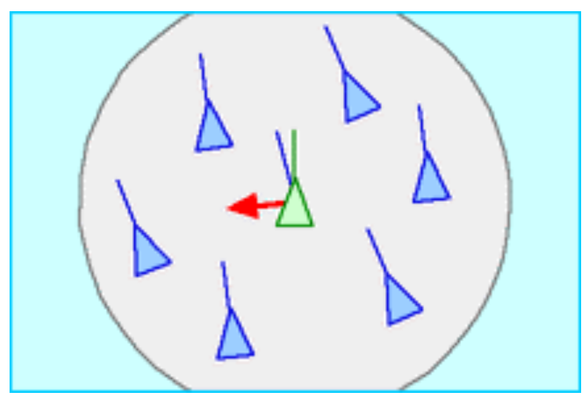
- attraction to center of neighbors
- repulsion from individual neighbors
- alignment toward average trajectory of neighbors



attraction



repulsion

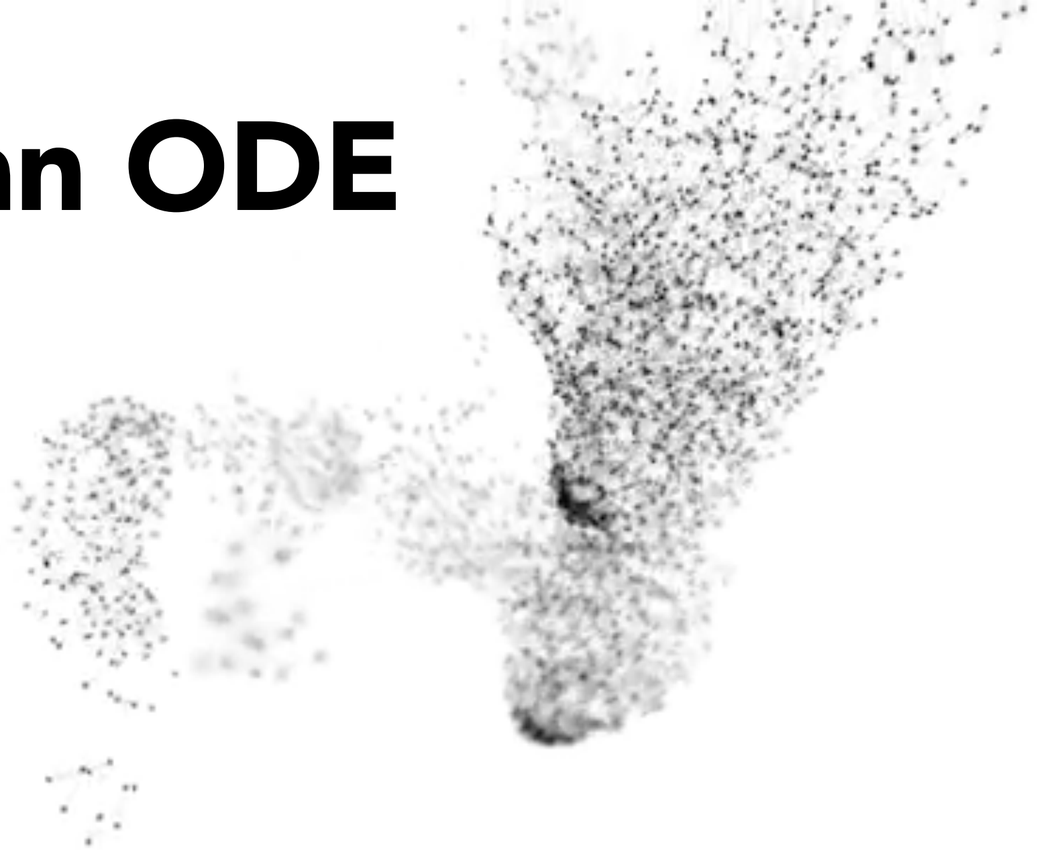


alignment

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

Simulated Flocking as an ODE

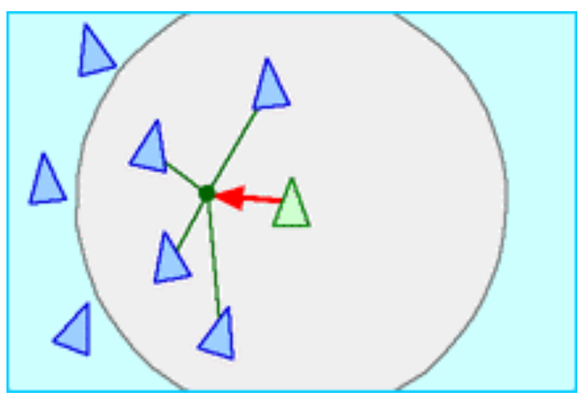


Model each bird as a particle

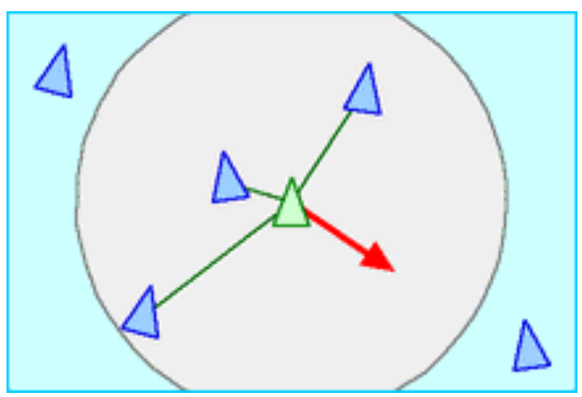
Subject to very simple forces:

- attraction to center of neighbors
- repulsion from individual neighbors
- alignment toward average trajectory of neighbors

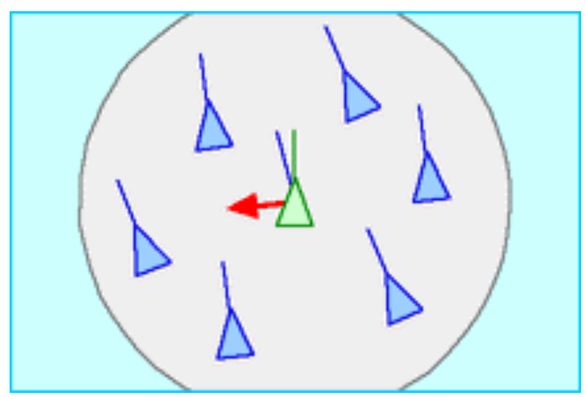
Simulate evolution of large particle system numerically



attraction



repulsion

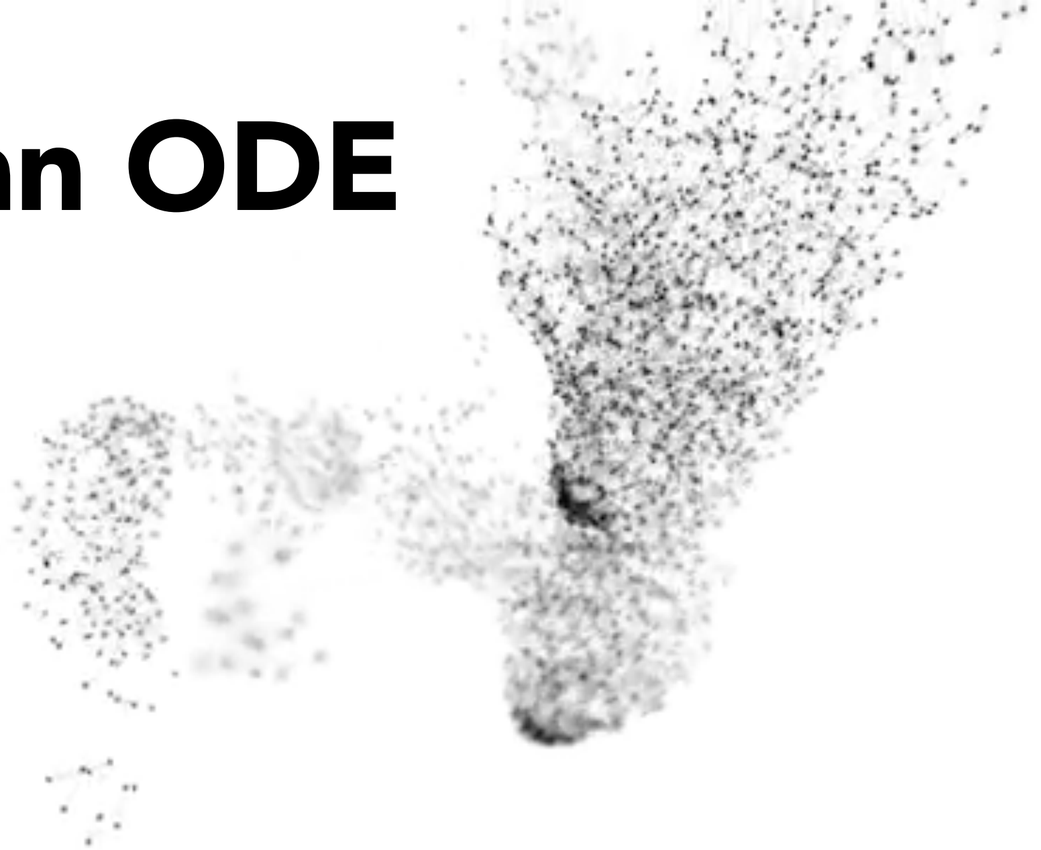


alignment

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

Simulated Flocking as an ODE



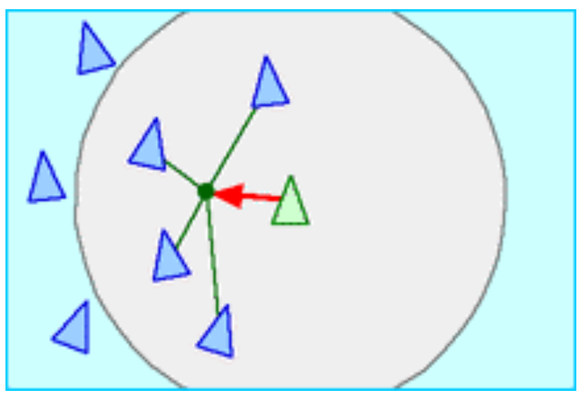
Model each bird as a particle

Subject to very simple forces:

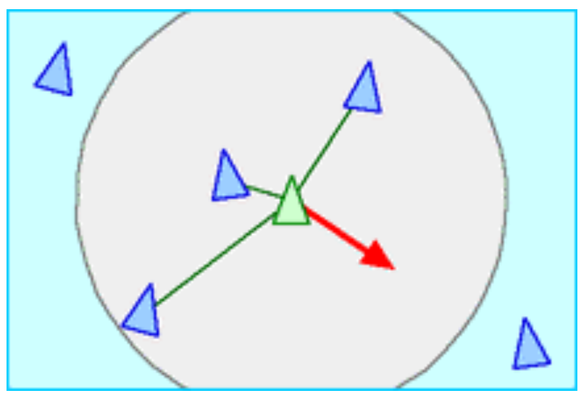
- attraction to center of neighbors
- repulsion from individual neighbors
- alignment toward average trajectory of neighbors

Simulate evolution of large particle system numerically

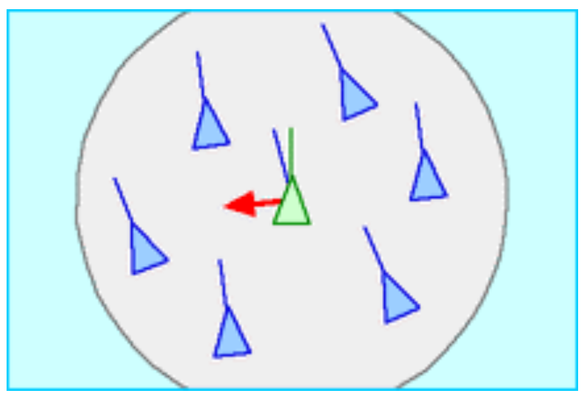
Emergent complex behavior (also seen in fish, bees, ...)



attraction



repulsion



alignment

Credit: Craig Reynolds (see http://www.red3d.com/cwr/boids/)

Slide credit: Keenan Crane

Example: Crowds



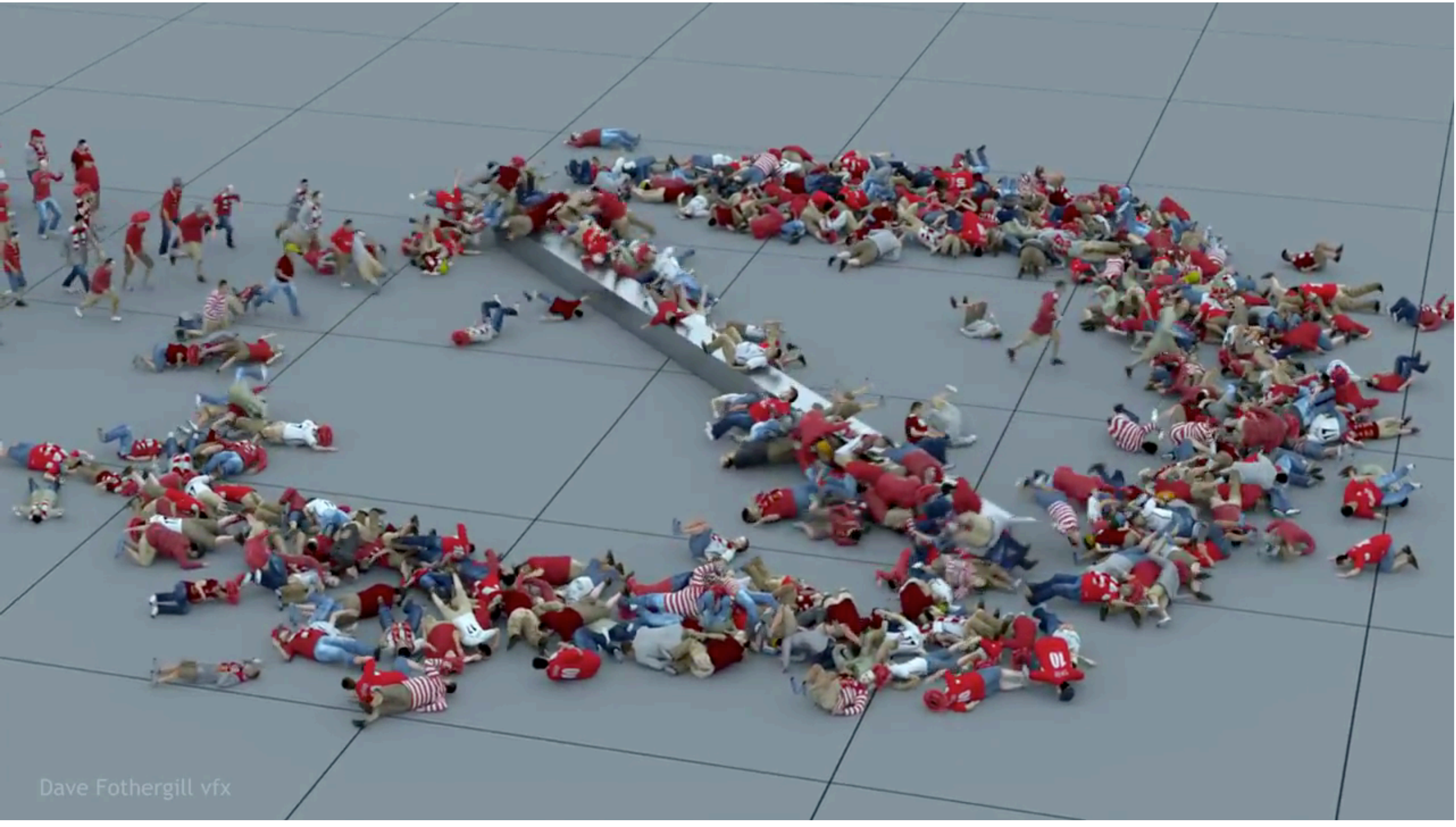
Where are the bottlenecks in a building plan?

Example: Crowds

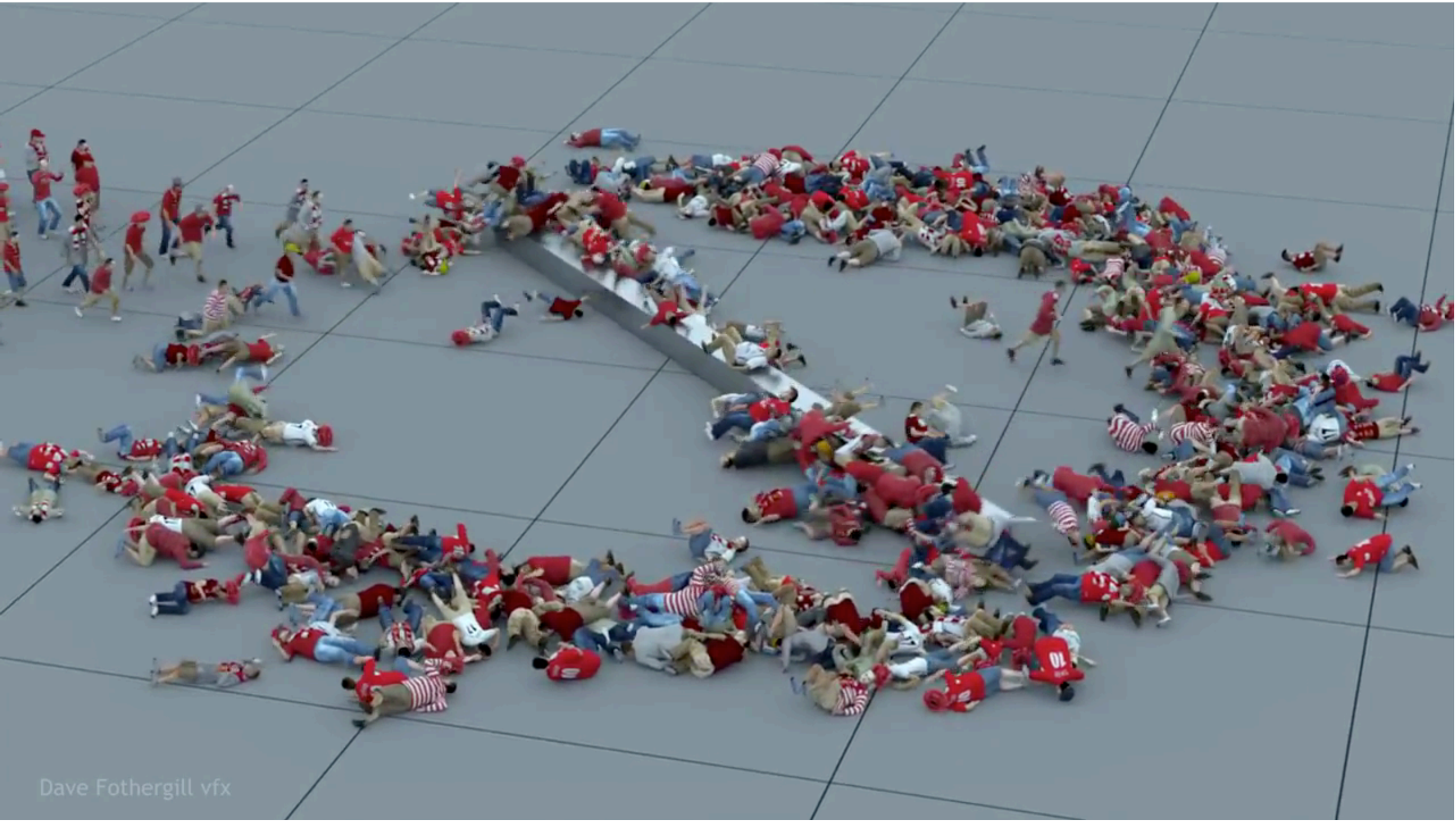


Where are the bottlenecks in a building plan?

Example: Crowds + “Rock” Dynamics



Example: Crowds + “Rock” Dynamics



Just Scratching the Surface…

Physical simulation is a huge field in graphics, engineering, science

Today: intro to particle systems, solving ODEs

Partial differential equations

- Diffusion equation, heat equation, …
- Used in graphics for liquids, smoke, fire, etc.

Rigid body

Simulation of sound

…

Things to Remember

Physical simulation = mathematical modeling of dynamical systems & solution by numerical integration

Particle systems

- Flexible force modeling, e.g. spring-mass sytems, gravitational attraction, fluids, flocking behavior
- Newtonian equations of motion = ODEs
- Solution by numerical integration of ODEs: Explicit Euler, Implicit Euler, Adaptive, Position-Based / Verlet
- Error and instability, methods to combat instability

Acknowledgments

This slide set contain contributions from:

- Keenan Crane
- Kayvon Fatahalian
- David Forsyth
- Pat Hanrahan
- Angjoo Kanazawa
- Steve Marschner
- Ren Ng
- James O’Brien
- Mark Pauly